

Foundation Models for EEG Signal Analysis

Master Project

Glenn Anta Bucagu

21-746-813

Alexandru Dimofte

23-733-819

Submitted on

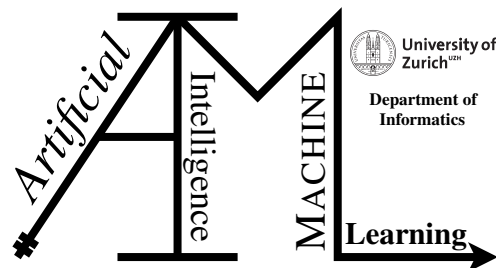
September 30th 2024

Master Project Supervisor

Pr. Luca Benini (ETHZ)

Master Project Co-Supervisors

Pr. Manuel Günther (UZH), Pr. Luc Van Gool (ETHZ), Dr. Yawei Li (ETHZ), Dr. Xiaying Wang (ETHZ), Thorir Ingolfsson (ETHZ)



ETH zürich

Integrated Systems Laboratory

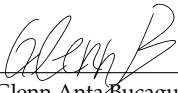


Declaration of Independence for Written Work

I hereby declare that I have **composed** this work independently and without the use of any aids other than those declared (including generative AI such as ChatGPT) – the use of generative AI to **improve** my composed work was permitted by the thesis supervisor. I am aware that I take full responsibility for the scientific character of the submitted text myself, even if AI aids were used. All passages taken verbatim or in sense from published or unpublished writings are identified as such. The work has not yet been submitted in the same or similar form or in excerpts as part of another examination.

30/09/2024, Zurich, Switzerland

Place, Date


Glenn Anta Bucagu

30/09/2024, Zurich, Switzerland

Place, Date


Alexandru Dimofte

Master Project

Authors: Glenn Anta Bucagu (glennanta.bucagu@uzh.ch)
Alexandru Dimofte (alexandru.dimofte@uzh.ch)

Project period: 11th March 2024 - 30th September 2024

Artificial Intelligence and Machine Learning Group
Department of Informatics, University of Zurich

Acknowledgements

The completion of this Master project would have been impossible possible without Pr. Luca Benini, Pr. Manuel Günther, Pr. Luc Van Gool, Dr. Yawei Li, Dr. Xiaying Wang and Thorir Ingolfsson. We thank Pr. Luca Benini and Pr. Luc Van Gool for their constructive feedback despite their busy schedules. We thank Pr. Manuel Günther for his comprehensive theoretical insights, particularly in the field of signal analysis. We thank Dr. Yawei Li, Dr. Xiaying Wang and Thorir Ingolfsson for giving us the opportunity to contribute to a field we knew very little about at the beginning. We thank them for their patience and unwavering support. We thank Dr. Li for his theoretical insights which directly led to the development of our split attention mechanisms. We thank Dr. Xiaying Wang and Thorir Ingolfsson for their EEG subject-specific advice, which was crucial in our model design, experiment design, data pre-processing as well as their tips regarding the ETH Zurich EULER computing platform.

Abstract

Electroencephalography (EEG) is a crucial tool for studying brain activity. Manual analysis and annotation of EEG signals can be time-consuming and prone to human error among other challenges. Supervised learning algorithms can capture the complex nature of EEG signals to some extent, but they require large quantities of high quality and correctly annotated EEG data, which is hard to procure. Recently, self-supervised algorithms have emerged as a potential solution to this problem, as they use more abundant unlabelled data. Current state of the art methods typically suffer from one of the following limitations: i) poor modelling of inter-channel correlations, ii) poor modelling of intra-channel temporal dynamics, iii) large resource requirements. Even when a model addresses these 3 limitations, it is pre-trained on private data and/or has no publicly-available implementation. This complicates scrutiny of existing models and development of new models. Our work aims to tackle all these challenges. We propose a Transformer-based model and self-supervised learning algorithm for EEG signal analysis. Our novel tokenization scheme, combined with self-attention, allows our model to learn intra-channel temporal dynamics as well as inter-channel spatial correlations. We propose new variants of self-attention coined split attention. This family of attention mechanisms splits the attention computation to intra-channel and inter-channel self-attention computation. As a result, split attention achieves significant reduction in memory usage (up to 4 times less) and computational time (up to 6 times faster). We pre-train and fine-tune our model on publicly available data. Our experiments show that our model is either superior or on par with most existing EEG foundation models in 2 downstream tasks, even with a significantly smaller pre-training corpus. In the rare cases where our model performs slightly worse than the state of the art, we posit that this is largely due to our relatively small pre-training corpus. We also provide a well-documented code base to facilitate reproducibility and further research.

Contents

1	Introduction	1
1.1	A Briefing on EEG Signals	1
1.2	Project Deliverables	2
1.3	Contributions and Work Distribution	2
1.4	Problem Statement	5
1.5	Related Work	6
1.5.1	EEG Foundation Models	6
1.5.2	Vision Foundation Models	10
1.5.3	Timeseries Foundation Models	11
2	Methodology	13
2.1	Datasets	13
2.1.1	Pre-training	13
2.1.2	Fine-tuning	14
2.2	EEG Signal Representation	14
2.2.1	Time Representation	14
2.2.2	Time-Frequency Representation	14
2.3	Tokenization	15
2.3.1	Channel-Mixing Projection	15
2.3.2	Channel-Independent Projection	16
2.3.3	Positional and Channel Encodings	16
2.3.4	[CLS] Token	17
2.4	Model Architecture	17
2.4.1	Standard Self-Attention	17
2.4.2	Split Attention	17
2.4.3	Handling the [CLS] Token	19
2.4.4	Theoretical and Empirical Analysis of Split Attention	20
2.5	Pre-training Strategy	24
2.5.1	Masking	24
2.5.2	Pre-training Style	24
2.5.3	Loss Function	25
2.6	Fine-tuning Strategy	25
3	The TimeFM Repository	27
3.1	What is TimeFM?	27
3.1.1	PyTorch Lightning	27
3.1.2	Hydra	27

3.2	Original State of TimeFM	27
3.3	Current State of TimeFM	28
3.3.1	Criteria	28
3.3.2	Schedulers	28
3.3.3	Datasets	28
3.3.4	Data Modules	28
3.3.5	Models	28
3.3.6	Model Heads	29
3.3.7	Pre-processors	29
3.3.8	Tasks	29
4	Experiments	31
4.1	Waveforms vs. Spectrograms	31
4.2	Tokenization Strategy	32
4.3	Attention Mechanisms	32
4.4	Comparisons to the State of the Art	32
5	Discussion	35
6	Conclusion	37
7	Limitations and Future Work	39
A	Pre-processing Details	41
B	Dataset Statistics	43
C	Ablation Studies	45
C.1	Incorporating Magnitude Information Into Spectrogram Tokenization	45
C.2	Hyperparameter Search	45
D	Training Recipes	47
E	Pre-training Reconstructions	49
F	What Did Not Work	51

Introduction

1.1 A Briefing on EEG Signals

An electroencephalogram (EEG) is a method that captures electrical activity in the brain (Teplan (2002)). EEGs have many applications including but not limited to disease diagnosis, medical monitoring, and brain-computer interfacing (Roy et al. (2019)).

There are several ways of collecting EEGs. Scalp EEGs involve placing electrodes on the surface of the scalp to measure electrical activity from the brain (Janiukstyte et al. (2023)). Intracranial EEGs (iEEGs) involve implanting electrodes directly on or in the brain to record electrical activity (Gherman et al. (2024)). The non-invasive nature of scalp EEGs makes them suitable for routine clinical use and widespread research applications, whereas iEEGs are typically reserved for specific clinical scenarios such as pre-surgical evaluation in epileptic patients. The human skull acts like a low pass filter and can therefore attenuate the signal provided by scalp EEGs (Ramon et al. (2009)). This is why iEEGs provide higher spatial resolution (< 1 cm) compared to scalp EEGs (~ 10 cm) (Burle et al. (2015)). The frequency content of each EEG signal type is also affected: scalp EEGs predominantly capture lower frequencies (typically 1-20 Hz since anything above is generally artifactual in standard clinical settings), while iEEGs can accurately capture frequencies in the range 40-150 Hz (Lachaux et al. (2012)). In contrast to iEEGs, scalp EEGs are also more susceptible to body movements like muscular twitches and blinking among others. This leads to iEEGs having 20-100 times higher signal-to-noise ratio (SNR) than scalp EEGs (Parvizi and Kastner (2018)). Our project is centred on scalp EEGs since their scalp EEG datasets are generally more abundant in the research community.

Scalp EEG signals exhibit several distinct mathematical characteristics. Viewed as multi-dimensional timeseries, they are inherently non-linear because the brain's electrical activity is generated by highly complex interactions among millions of neurons. They are also highly correlated due to the interconnected nature of brain regions and the shared pathways that influence their activities (Rashid et al. (2016)). Moreover, EEG signals are non-stationary (both in the weak and strong sense), meaning their statistical properties (e.g. statistical moments, autocovariance) change over time (Klonowski (2009)). These mathematical properties can make it difficult to model scalp EEG signals, especially using classical timeseries forecasting models such as variations of ARIMA and ETS (Brockwell and Davis (2016)).

Deep learning (DL) methods have shown significant promise in computer vision (Dosovitskiy et al., 2021), natural language processing (NLP) (Devlin et al., 2019) and biomedical science (Lee et al., 2024) among other fields. More recently, foundation models have emerged as leaders in a

variety of tasks (Bommasani et al., 2022). These models generally require very large pre-training corpora (e.g. sizeable subsets of the internet in the case of state of the art NLP models). They are then fine-tuned on specific downstream tasks. In the context of EEG signals, this formula is difficult to replicate. Large EEG datasets are difficult to compile because of time, privacy and cost constraints (Boudewyn et al. (2023)). Moreover, labelling is difficult to do consistently since it requires highly trained and experienced medical practitioners. Even when such practitioners are available, their annotations can be subject to human errors (Diachenko et al. (2022)). Even if large amounts of correctly labelled EEG datasets were abundantly available, large and deep models would likely overfit the EEG data, particularly in the presence of high inter-subject variability (Schirrmester et al. (2017); Kostas et al. (2021)). A popular solution to these problems is self-supervised learning (SSL). This method involves collecting large unlabeled data and producing supervisory signals to a model such as to construct meaningful and useful representations of the data.

1.2 Project Deliverables

Our Master project has the following objectives:

1. To develop a new foundational model tailored explicitly for EEG signal analysis.
2. To utilize extensive, publicly available EEG data to pre-train the model, ensuring a comprehensive and diverse data foundation.
3. To evaluate the model's effectiveness in real world EEG tasks.
4. To provide an efficient, well-organized and well documented implementation such that the work can be reproduced and built upon.

1.3 Contributions and Work Distribution

To complete the project deliverables, we aimed to keep an overall 50-50 workload split. In this section we highlight our individual contributions to the project.

Weeks 1 to 2: Project Setup and Literature Review

- Both:
 - Established project structure, scheduled weekly meetings, and reviewed key literature on EEG foundational models.
 - Set up computing environments in the Computer Vision Lab computing infrastructure and familiarized ourselves with the TimeFM repository.

Weeks 3 to 5: Initial Model Integration and Dataset Preparation

- Alex:
 - Integrated BrainBERT into TimeFM
 - Added first version of masked spectrogram reconstruction task into TimeFM.

- **Glenn:**
 - Downloaded, pre-processed and created specific dataset class for the CHB-MIT dataset ([Shoeb \(2010\)](#)).
 - Conducted literature review on EEG encoding methods (spectrograms, waveforms, patching methods)
- **Both:**
 - Identified theoretical limitations in current state of the art EEG foundation model tokenization and proposed a new alternative..

Weeks 6 to 8: ChannelViT and Hierarchical Channel Sampling

- **Both:**
 - Decided on our first candidate encoder and decoder. The former based on ChannelViT ([Bao et al., 2024](#)), and the latter based on BrainBERT ([Wang et al., 2023](#))
 - Ran tests using our proposed model, and the data introduced during this week.
 - Implemented ChannelViT, and hierarchical channel sampling (HCS) in TimeFM.
- **Alex:**
 - Implemented a candidate decoder based on BrainBERT.
- **Glenn:**
 - Downloaded, pre-processed and implemented a dataset class for the TUSL dataset.

Weeks 9 to 12: Pre-training and Fine-tuning Pipeline

- **Both:**
 - Completed the first pre-training run on TUSL+TUEP+TUSZ and fine-tuned the model on TUAB.
 - Migrated to EULER platform.
- **Alex:**
 - Implemented the fine-tuning Task in TimeFM.
 - Added positional and channel-wise encodings.
 - Implemented masking as used in BrainBERT.
 - TimeFM refactoring: Created separation between models and model heads, standardized outputs and inputs of each module group such that all modules are mutually compatible.
- **Glenn:**
 - Download and pre-processing of TUAB dataset.
 - Communicated with Dr. Joseph Picone, administrator of the Temple University Hospital EEG data repository regarding dataset access and structure.
 - Implemented PyTorch dataset for pre-training which concatenates an arbitrary number of PyTorch datasets.
 - Added SPaRCNet and simple convolution models to test our pipeline's integrity.
 - Pre-processed .edf files and converted them to .npy files for quicker data loading.

Weeks 13 to 15: Model Debugging and Memory Issues

Several bugs appeared this week, which pointed at the fact that our model was not learning. Moreover, we had difficulty launching jobs in EULER, since our jobs were not using all the GPUs we requested.

- **Both:**

- Tested different fine-tuning strategies and experimented with masking and patching methods.

- **Alex:**

- Integrated LaBraM model into TimeFM to test pipeline with a different model.
- Investigation of memory usage of our model candidate and LaBraM at different layers of abstraction (e.g. entire model, attention mechanism only, etc)
- Fixed our SLURM sbatch script such that we can use several GPUs on one training run.
- Removed unwanted activation in the MLP prediction layer.
- Cleaned and standardized entire code-base for better readability.

- **Glenn:**

- Recreated SpaRCNet results in TimeFM and ran controlled experiments in order to find bug.
- Implemented FLOPs calculation in the pipeline.
- Identified frequency-wise spectrogram normalization as a limitation for scalp EEGs tokenization. Implemented channel-wise normalization. This, in combination with the removal of the unwanted activation led to proper fine-tuning of our model.
- Identified the mismatch between masking and tokenization as primary cause of poor model performance.

Weeks 16 to 22: Split Attention Mechanisms, ViTMAE and SimMiM

- **Alex:**

- Developed time-wise patching.
- Implemented ViTMAE for waveforms.
- Implemented Alternating and Bottleneck split attention mechanisms.
- Conducted theoretical analysis of the split attention mechanisms.
- Added functionality for pre-training and fine-tuning on waveforms.
- Validated pipeline on CIFAR10 (since we know how model should behave)
- Conducted first MAE pre-training and fine-tuning experiments on waveforms.

- **Glenn:**

- Literature review on pre-training datasets and downstream task datasets.
- Implemented SimMiM and ViTMAE (linear spectrogram patch embedding, positional and channel encodings, encoder, decoder, loss computation) for spectrograms.
- Generalized loss function: added possibility for L1, L2 or Smooth L1 and added loss component on visible patches

- Conducted preliminary pre-training and fine-tuning experiments on spectrograms using SimMiM.
- Improved pre-processing pipelines and performance metrics, moved STFT computation to the GPU
- Implemented more sophisticated training recipes including cosine scheduler, layer-wise learning rates and label smoothing.

Weeks 23 to 25: Final Model Refinements and Pre-training

- **Alex:**
 - Improved waveform pre-processing, conducted detailed patch size experiments.
 - Fixed bugs in layer-wise learning rate decay during fine-tuning.
 - Optimized of hyperparameters for fine-tuning.
- **Glenn:**
 - Noise augmentations for spectrograms and waveforms.
 - Time-bin patching functionality which aggregates time-steps.
 - Added and pre-processed Neonate dataset
 - Proposed Two Axis split attention mechanism
 - Proposed normalization of waveforms to stabilize training on waveforms.
- **Both:**
 - Experimented with split attention variations

Weeks 26 to 29: Final Experiments and Report Writing

- **Alex:**
 - Pre-training and fine-tuning validation experiments on waveforms
 - Edited the project report.
- **Glenn:**
 - Pre-training and fine-tuning validation experiments on spectrograms
 - Drafted project report.
- **Both:**
 - Conducted memory and runtime comparison for different attention mechanisms.

1.4 Problem Statement

We formalize SSL for EEG signals as follows: Suppose we have a dataset $D = \{X_1, X_2, \dots, X_n\}$, where $X_i \in \mathbb{R}^{C \times T}$, $i = 1, \dots, n$ denotes an EEG sample with C channels and T time points. The objective of self-supervised pre-training is to learn a non-linear embedding function to map each EEG sample in D to an embedding dimension d_e of our choice. This training is self-supervised because no natural labels are used. We artificially create supervisory signals to train the model (see section 2.5). We can evaluate the quality of pre-training using fine-tuning; this is where the pre-trained model is further trained and evaluated on a smaller labelled dataset under a classical supervised setting. In our setup, the pre-training and downstream datasets are disjoint.

1.5 Related Work

1.5.1 EEG Foundation Models

Our work builds on the rich literature of EEG foundation models. Similarly, to modern day natural language processing, Transformers dominate the field of EEG foundation modelling.

BENDR (Kostas et al. (2021)) adapts wav2vec 2.0 (Baevski et al. (2020)). The input EEG waveforms are heavily downsampled via a stack of short-receptive-field 1D convolutions. These convolutions map the waveforms to a sequence of tokens that is provided as input to a standard stack of Transformer encoder layers. Positional information is encoded using an additive grouped convolution layer. This approach has the advantage of making the model sequence length-independent, but it comes at the expense of not properly encoding positional information for short sequences. To pre-train BENDR, the authors follow a Masked Autoencoding (MAE) scheme, where tokens are randomly masked and reconstructed by the model. As the first pre-trained EEG foundation model, BENDR is very promising. However, there are a few limitations in the model. BENDR does not properly account for spatial correlations that come from different channels. We note the lack of channel-specific embeddings, meaning the model cannot reliably discern different channels. Finally, the authors conduct ablation studies that suggest BENDR may be retaining redundant temporal information.

BIOT (Yang et al. (2023)) tackles some of the challenges presented by BENDR. Each waveform is sliced into non-overlapping segments. A Fast Fourier Transform is computed over each segment to extract frequency information and provide a segment embedding. Learned channel and spatial embeddings are added to each segment embedding to construct what becomes the input tokens. By incorporating rank-d attention approximation, BIOT leverages linear attention modules. During training, tokens are randomly dropped (not masked) from each training example, thereby creating a corrupted signal. BIOT is then optimized via a contrastive loss. Although the linear attention module is more efficient than the standard quadratic attention module, it leads to lower performance both in pre-training and downstream.

Unlike BIOT, which uses only a frequency representation of EEG signals, BrainBERT (Wang et al. (2023)) uses a time-frequency representation, either via a Short-Time Fourier Transform (STFT) (Sejdić et al. (2009)) or a Superlet (Moca et al. (2021)) Transform. Waveforms are first sliced into non-overlapping segments. Then, either a STFT or Superlet Transform is applied to each channel individually. 1 token is the linear projection of 1 time bin from 1 spectrogram. This gives the tokens a natural sequential ordering. Sinusoidal positional embeddings are added to each token. Each sequence of tokens is then passed through a standard stack of Transformer layers followed by a lightweight linear spectrogram prediction head. BrainBERT is pre-trained via MAE: variable width time bins and frequency bins are randomly masked in each spectrogram. BrainBERT is then optimized via an L_1 reconstruction loss. It is clear that a time-frequency representation provides a model with a rich representation of each waveform. However, BrainBERT's primary limitation is that it does not consider multi-channel inputs. In other words, a single forward pass only considers 1 channel. This means the model does not learn inter-channel spatial correlations, which are key in EEG modelling. It also results in a complex fine-tuning process where the practitioner must manually select the most "important" channels. Ideally, the attention mechanism should do this automatically. Moreover, BrainBERT is pre-trained and fine-tuned on private iEEG data. It is unclear whether this performance translates to scalp EEG data.

Brant (Zhang et al. (2023)) follows the typical approach of slicing the EEG waveforms into non-

overlapping segments. Each segment is then augmented by adding its absolute spectral power. This is a learned frequency encoding. This tokenization process effectively constructs a time-frequency representation of the original signal. Brant then has a separate temporal and spatial encoder. These are both stack of Transformer layers with different purposes. The temporal encoder is responsible for learning any temporal relationships between tokens. The spatial encoder is then responsible for learning inter-channel spatial relationships from the temporal hidden representations. A lightweight spectrogram prediction head is placed after the spatial encoder. Brant is pre-trained by randomly masking tokens, reconstructing them and minimizing the L_2 loss between the input and reconstruction. Brant addresses some of BrainBERT’s limitations; inter-channel spatial relationships are explicitly accounted for by the model. However, Brant is limited by the fact that it is only pre-trained on private iEEG recordings. Brant (505.69 M parameters) is approximately 11.7 times larger than BrainBERT (43.18 M parameters). Even though Brant compares its downstream performance to BrainBERT, it is unclear whether the improved performance is primarily due to the differences in model architecture and tokenization or due to the model size.

Brant-2 (Yuan et al. (2024b)) aims to scale up and improve upon Brant. While the tokenization and model architecture are largely unchanged, there are a few notable differences. While Brant projects each patch to an embedding dimension using a simple linear projection, Brant-2 performs this projection using a small convolutional network. Before the tokens are fed to the temporal encoder, Brant-2 introduces a data augmentation module. This augmentation module randomly resamples each signal patch to provide the model with a variety of sampling rates and randomly drops and reshuffles the channels to provide the model with different channel configurations. While Brant uses MAE as a pre-training task, Brant-2 combines MAE and out-of-sample forecasting jointly. Brant-2 (1B parameters) is much larger than Brant (505.69 M parameters), but it is also pre-trained on a much larger dataset (over 27000 hours of recordings from over 15000 different patients). While Brant-2 is the largest EEG model to date (both in terms of model size and pre-training dataset size), any comparisons with previous models has to be done at size and pre-training data size parity. For full transparency, we note that Brant-2 has not been published in a peer-reviewed conference or journal. A member of our group at ETH Zurich tried to replicate the results of Brant-2 unsuccessfully. The authors of Brant-2 were unresponsive to our inquiries.

Inspired by VQ-VAE (van den Oord et al. (2017)), EEGFormer (Chen et al. (2024)) applies Vector-Quantized pre-training to EEG modelling. The EEG waveforms are sliced into non-overlapping patches. A Fourier transform is used to compute frequency amplitude as the effective input features which are then projected to an embedding dimension. Positional embeddings are added to this. The resulting tokens are passed through a Transformer encoder model. A vector quantizer and quantized codebook are used to map these patch representations to discrete tokens. The vector quantizer looks up the nearest neighbor in the quantized codebook for each patch representation. These discrete tokens are then passed to a Transformer decoder model, which reconstructs the input patches. EEGFormer’s pre-training task is autoencoding optimized via an L_2 loss (with additional regularization to account for the straight-through gradients used in the vector quantizer). EEGFormer comes in 3 different sizes which facilitates comparison with existing models. EEGFormer is pre-trained on publicly available scalp EEG recordings, namely the Temple University Hospital (TUH) EEG corpus. The primary limitation of EEGFormer is the lack of an ablation study to scrutinize the importance of the vector quantizer. The vector quantizer also introduces extra computational overhead, although the authors do not provide any statistics (i.e. additional floating point operations (FLOPs) or trainable parameters). The authors do not mention any channel embeddings, which means the Transformer encoder and decoders have no way of discerning different channels. EEGFormer does not have any publicly available code implementation and the authors were unresponsive to our inquiries.

LaBraM (Jiang et al. (2024)) adapts the vector quantizer used by EEGFormer. Instead of the standard tokenization discussed in previous models, LaBraM is the first EEG foundation model to introduce a learned tokenizer. EEG waveforms are sliced into non-overlapping segments. The authors define a quantized codebook. A quantizer maps the patch representations into neural codebook embeddings by using the quantized codebook as a lookup table. The neural codebook embeddings are then fed into a stack of Transformer decoder layers. The neural tokenizer is trained by reconstructing the Fourier spectrum (amplitude and phase) of the input patches. This tokenizer is trained by minimizing the L_2 loss computed on the amplitude and phase separately (plus additional regularization to account for the straight-through gradients used in the vector quantizer). The actual LaBraM model is trained as follows: EEG waveforms are split into non-overlapping segments which are then fed into several temporal convolutional blocks to obtain patch embeddings. Temporal and spatial (or channel) embeddings are then added to the patch embeddings. The resulting tokens are fed into a stack of Transformer encoder layers. The resulting output is passed to a token prediction head. To pre-train LaBraM, tokens are randomly masked and then fed to the model. By minimizing the negative log conditional probability of predicting the correct tokens given the input masked tokens, LaBraM learns meaningful representations of EEG signals. The ground truth tokens are assumed to be those produced by the trained neural tokenizer. LaBraM has the advantage of being pre-trained on many different publicly available EEG datasets, with different subjects, numbers of channels, measuring devices etc. Indeed, LaBraM demonstrates state of the art performance on 4 different downstream tasks. There are however a few limitations. Training a neural tokenizer conjures additional compute resources. These additional resources could be used to train a significantly larger version of a model with a standard non-learned tokenizer. LaBraM provides no ablation study to highlight the importance of this learned tokenizer. LaBraM makes no comparisons to other competitive models like BrainBERT and Brant. From a technical point of view, the neural tokenizer loss involves an L_2 component computed on the signal phase. The phase is a cyclical variable so an angular loss might be more appropriate.

In Neuro-GPT (Cui et al. (2024)), EEG waveforms are sliced into non-overlapping segments. These segments pass through a convolutional block that contains both spatial and temporal convolution filters. The resulting features are average pooled and then passed onto Transformer encoder layers. The self-attention mechanism combined with convolution encodes the spatio-temporal features of each training example. Neuro-GPT introduces a novel causal masking scheme in which several copies of the same training example are passed to a GPT decoder model in each forward pass. In the first copy, only the first token is visible to the model, in the second copy only the first two tokens are visible and so on until only 1 token is invisible to the model. Neuro-GPT is pre-trained by minimizing the L_2 loss between reconstructed patches and input patches. Neuro-GPT does not explicitly mention how positional and channel information is encoded into the model. This seems like a necessary process especially if the model needs to be trained on datasets with different channel configurations. Causal masking may also be suboptimal since the model may learn to attend to earlier tokens. Methods like LaBraM, Brant and Brant-2 use a random unstructured masking pattern, which doesn't bias any token in particular.

Mentality (Panchavati and Speier (2024)) is the first model not to use the Transformer architecture in EEG signal modelling. Instead, Mentality uses Mamba blocks (Gu and Dao (2024)). Mamba is a selective state model that was recently proposed as an alternative to Transformers. Unlike the attention mechanism, which has quadratic time and space complexity in the sequence length, Mamba boasts linear scaling in the sequence length. In principle, this means that for the same compute budget, Mamba yields faster inference and less memory requirements than a Transformer model. We must note that at the time of writing, Mamba is less than a year old so it has

not been thoroughly investigated in practice like the Transformer has. To pre-train Mentality, EEG waveforms are sliced into non-overlapping segments. These are passed onto a 1D convolutional layer with a kernel size equal to half the sampling rate of the signal to learn “frequency-based” filters from each channel. A channel-mixing linear layer is then used to learn any inter-channel spatial correlations. This is followed by several Mamba blocks to learn a state space representation of the data, emphasizing the temporal dynamics. The output is then downsampled through a U-Net (Ronneberger et al. (2015)) block that is composed of a double convolution and mean pooling. The resulting sequence is then passed onto a different set of Mamba blocks to obtain a downsampled hidden representation of the data. To reconstruct the original signal, the hidden representation is upsampled through repeated Mamba-transpose convolution and double convolution blocks. Inspired by U-Net, Mentality uses residual connections. Mentality incorporates time and frequency information in the loss term by computing an L_2 loss between input and output signal as well as a spectral loss on the Fourier representations. Although Mentality presents a novel approach to self-supervised EEG signal modelling, it is not rigorously tested. There is only a single baseline on seizure detection and this is not compared to any other model. Moreover, the channel-mixing projection is one that is sub-optimal as discussed by PatchTST (Nie et al. (2023)) for multi-channel timeseries and ChannelViT for multi-channel images. A better alternative is to have a channel-independent approach where each token only contains information about a single channel.

Hussein et al. (2022) proposed MultiViT - a method in which a single ViT encoder is trained for each channel of EEG spectrograms. The final spectrogram representation is derived by aggregating the output [CLS] tokens across all single-channel ViTs. An MLP is then attached to these aggregated features to predict the spectrogram label. We note that MultiViT uses direct supervised training (seizure prediction). MultiViT calls for huge computational resources and suffers from the lack of cross-channel attention. The huge computational resources required to train this model make it inappropriate for large scale SSL, specifically for multi-channel EEG signals.

Foumani et al. (2024) recently proposed EEG2Rep, a model that uses context-driven target prediction for EEG data to overcome limitations of MAE. In this regime, a model is trained to predict the representations of the original unmasked training data based on an encoding derived from the masked sample in abstract representation space. The authors suggest that using abstract targets instead of raw targets (like in MAE), noise and unnecessary raw-level features can be eliminated. EEG2Rep is broken up into 3 phases. First, a target network takes the full unmasked input embeddings and produces semantically rich representations via the self-attention mechanism. Secondly, a context network generates representations only for visible patches. Finally, a predictor network leverages the output of the context network and randomly chosen masked tokens to regress the output of the target network. The predictor network is a 4-layer cross-attention transformer. The model is trained by minimizing the L_2 computed between the predictor outputs and the target network outputs in the masked positions. The authors also propose a novel tokenization method involving a 3-layer convolutional network. Depth-wise convolutions are used to capture inter-channel correlations and a linear spatial filter is used to amplify the SNR in each EEG patch. The primary limitation is that by using a channel-mixing tokenization approach, inter-channel correlations are only captured in the 3-layer convolutional network as opposed to the self-attention or cross-attention mechanisms in the context, target and predictor networks. This is an information bottleneck.

1.5.2 Vision Foundation Models

Some of our work builds upon state of the art vision foundation models. In particular, we use similar tokenization methods, architectures and loss functions with some domain-specific modifications for our data.

ViT (Dosovitskiy et al. (2021)) is the first convolution-free use of the Transformer architecture for computer vision tasks. All experiments are conducted on RGB images such as ImageNet (Deng et al. (2009)), CIFAR-100 (Krizhevsky (2009)) and VTAB (Zhai et al. (2019)). To tokenize an RGB image, each image channel is split into non-overlapping square patches. To obtain each token, ViT uses a channel-mixing projection layer. This means each token contains information from the same patch position aggregated over all channels. Learnable positional embeddings are added to the resulting tokens. Finally, a single [CLS] token is pre-pended to the input sequence, which is then passed to standard Transformer encoder model. ViT uses supervised pre-training. The [CLS] token effectively contains a representation of all other tokens so one can append an MLP head to it to perform classification downstream.

SimMiM (Xie et al. (2022)) extends the work done by ViT to self-supervised pre-training. The tokenization and model architecture remains the same. During pre-training, the majority of tokens (it is shown that the optimal masking ratio is around 75%) are masked and replaced by a shared learnable [MASK] token. The new input sequence (which now contains a [CLS] token, visible tokens and [MASK] tokens) is passed onto a Transformer encoder model. The resulting hidden representation is passed onto a single linear prediction layer which reconstructs the all the masked tokens. The model learns by minimizing the L_1 loss computed between the original pixels and the model reconstructions. Note that in this process, the loss is only computed on the masked patches. Just like ViT, SimMiM is constrained to RGB images.

ViTMAE (He et al. (2021)) proposes an alternative self-supervised pre-training framework. Using the same tokenization as SimMiM, ViTMAE proposes separating the encoding and decoding tasks. In particular, masked tokens are not used in the decoder; the Transformer encoder model only receives shuffled visible tokens (and the [CLS] token) as input. These encoded representations are then unshuffled, and combined with learnable [MASK] tokens in the positions of the masked tokens. A smaller Transformer decoder model receives this as input and reconstructs the original sequence. This pre-training process minimizes the L_1 loss computed between the original pixels and the model reconstructions. Just like in SimMiM, the loss is only computed on the masked patches. This work is constrained to RGB images. Interestingly, ViTMAE and SimMiM yield similar downstream performance on ImageNet-1K under the same experimental conditions.

ChannelViT (Bao et al. (2024)) extends the work of ViT to up to 8 channels. The tokenization no longer uses a channel-mixing projection. Instead, the authors propose to use channel-independent projection. This way, each token contains information about a single channel. Inter-channel correlations are learnt via the attention mechanism which is computed pairwise on all tokens. Using supervised pre-training on ImageNet, the authors show that their model can improve upon the standard ViT framework. The channel-independent tokenization increases the number of tokens in each sequence by a factor equal to the number of channels. To mitigate the added computational overhead in the attention module, the authors propose a hierarchical channel sampling augmentation in pre-training. In this augmentation, channels are randomly dropped and shuffled around. ChannelViT demonstrates that their approach works well between 3 and 8 channels. In our work on spectrograms, we propose using ChannelViT's channel-independent tokenization applied to spectrograms with up to 23 channels. We note that ChannelViT only tests their framework under a supervised training regime. We propose extending ChannelViT's approach

to include self-supervised pre-training.

There are plenty of other high performing vision foundation models (Bao et al. (2022); Caron et al. (2021)) but they are not directly relevant to our work.

1.5.3 Timeseries Foundation Models

PatchTST (Nie et al. (2023)) applies the ViT framework to multi-channel timeseries. Similar to image patching, PatchTST segments the timeseries into patches. This has the benefit of preserving local semantic information in each embedding, quadratically reducing the computational and memory usage of attention maps and attending to a longer history of a given timeseries. PatchTST introduces channel-independent tokenization for timeseries, such that each token only contains information about a single channel. The authors use MAE to pre-train their Transformer encoder model, thereby minimizing the L_2 loss computed between input masked patches and their reconstructions. The primary limitation is that same embeddings are shared across each channel so the model cannot learn any inter-channel correlations. This is a sub-optimal solution in the context of EEG modelling, where inter-channel correlations are critical in downstream performance.

There are plenty of other high performing foundation models for timeseries (Chang et al. (2024); Jin et al. (2024); Gu et al. (2022)) but they are not directly relevant to our work.

Methodology

2.1 Datasets

When pre-training a foundation model for EEG signal analysis, there are several criteria that must be satisfied. Firstly, the pre-training dataset needs to be large enough. This is not only because the model itself is large (millions to billions of trainable parameters), but also because the model needs to capture general patterns in order to be truly transferable downstream (El-Nouby et al., 2021). There exist neural scaling laws, specifically for large language models in natural language processing (Kaplan et al., 2020). Given the size of the Transformer model being pre-trained, these scaling laws can inform how large the pre-training dataset needs to be. We note that there is no evidence that these exact scaling laws equally apply to EEG foundation models. More research in the field is needed. The pre-training data also has to be of high quality. Although scalp EEG signals have relatively low SNR compared to iEEG, respecting adequate clinical settings and using appropriate instruments is still important to maintain good quality signals. The dataset also needs to be highly variable. Low variability can introduce bias, which in turn leads to poor downstream generalizability. Ideally, one should observe a wide variety of subject profiles with varying cognitive processes and pathologies, multiple recording sessions per subject, different signal recording devices with different electrode positions, different numbers of channels and different sampling rates. In practice however, of all the models discussed in section 1.5, only LaBraM satisfies the variability criterion. This comes at the cost of having to train a neural tokenizer as well as pre-training a foundation model.

2.1.1 Pre-training

Continually collected since 2002, the TUH EEG corpus (Obeid and Picone, 2016) is perhaps the world’s largest publicly available corpus of clinical EEG data. It contains over 60,000 EEG recordings totalling over 30 years of recordings (when summed over all channels), from over 10,000 unique subjects. Among these subjects there is an approximately 50-50 male-female split, with ages ranging from 1 to 90 years old. The mean age is approximately 50 years old. All channel montages follow the International 10-20 system for electrode placement. We use this dataset for several reasons: firstly, it satisfies the criteria outlined in earlier in section 2.1 and secondly, it is used by several existing EEG foundation models discussed in section 1.5, thereby facilitating fair comparisons. In particular, we select several disjoint subsets of the TUH EEG corpus that each depict different pathologies: TUAB (anomaly detection), TUEV (event classification), TUSZ (seizure detection), TUEP (epilepsy classification), TUSL (slowing event detection) and TUAR (artifact classification). We pre-train our model on TUEV, TUSZ, TUEP, TUSL and TUAR. Statistics regarding our pre-training datasets can be found in appendix B.

2.1.2 Fine-tuning

To measure the downstream performance of our model, we fine-tune it on 2 different downstream tasks. First, we fine-tune it on TUAB. The objective is for the model to take multi-channel EEG signals as input and classify them either as normal or abnormal. The labels are curated and verified by qualified clinicians at or working in collaboration with TUH. There is an approximately 50-50 split of normal and abnormal EEG signals.

We then fine-tune our model on the Neonate seizure dataset (Stevenson et al., 2019). The objective is for the model to take multi-channel EEG signals as input and detect seizures. The particular challenge with this dataset is that it only records brain activities of 79 human neonates (i.e. newborn infants up to 1 month old). We do not have these subject profiles in our pre-training dataset so the goal is to examine how generalizable our model can be in comparison to existing EEG foundation models. Seizure annotations are independently provided by 3 clinicians. Whenever there are disagreements, we consider the majority vote. This dataset is heavily imbalanced, with only 13% of examples containing seizures.

Detailed statistics regarding our fine-tuning datasets can be found in appendix B.

2.2 EEG Signal Representation

There are many key decisions to be made when constructing a foundation model. The first decision is how to represent our data within the model. EEG recordings are usually available in .edf (European Data Format) or .gdf (General Data Format) as waveforms. We compare time and time-frequency representations because they are the most common in literature and they are yet to be directly compared for the same foundation model. Balancing time and frequency resolution is a crucial trade-off when choosing between these two representations.. The standard deviations of the time $\sigma_{energy,t}$ and frequency $\sigma_{energy,f}$ energy cannot both be arbitrarily reduced. This is formalized in the Heisenberg-Gabor uncertainty principle (Feichtinger and Strohmer (1998)):

$$\sigma_{energy,t} \cdot \sigma_{energy,f} \geq \frac{1}{4\pi} \quad (2.1)$$

We investigate EEG signal representations that handle this trade-off differently.

2.2.1 Time Representation

Many existing EEG and timeseries foundation models adopt a pure time representation of signals. Representing the EEG signals as waveforms has several advantages. Firstly, this yields the maximum temporal resolution. It is also relatively computationally efficient (when compared to an additional Fourier or Wavelet transform for example), and prevents any information from being lost in such transforms. Despite our use of bandpass and notch filters for waveform pre-processing, EEG waveforms can still contain significant noise. It is up to the model to learn to distinguish signal from noise. The high temporal resolution comes at the cost of frequency resolution; directly using waveforms means the model may not optimally capture frequency information which is key in EEG-related tasks.

For a detailed account of our waveform pre-processing, consult Appendix A.

2.2.2 Time-Frequency Representation

In contrast to a pure time representation, we propose a time-frequency representation. Since a Fourier Transform only gives frequency information, we propose computing the Fourier trans-

form over overlapping time windows (i.e. using an STFT representation). However, STFT requires that the input signal be stationary. Although EEGs are non-stationary, we can assume they are quasi-stationary over short time windows. This approximation generally yields good results in EEG signal analysis (Klonowski, 2009). STFT tackles the time-frequency resolution trade-off by providing the same time resolution for all frequencies. There are other transforms such as Superlet and Gabor (Lee (1996)), but we select STFT because it is the most well tested in scalp EEGs (Wang et al., 2023; Hussein et al., 2022). For further research, one may want to use a transform that provides better time resolutions at lower frequencies since this is where most of the EEG signal is localized. The time-frequency representation has the advantage of providing the model with explicit temporal and frequency information to learn from. However, they require more compute. Multi-channel waveforms can be represented by 2D tensors (number of channels $\times$ signal length), whereas multi-channel spectrograms have to be represented by 3D tensors (number of channels $\times$ frequency bins $\times$ time bins). Recall that spectrograms are complex-valued. Spectrogram-based models both for EEG signals and audio signals (Huang et al., 2022) typically only preserve the magnitude information from spectrograms. Discarding the phase information inevitably leads to loss of information. We carried out preliminary experiments, by training our model from scratch on some TUAB. We found that including phase information yields twice as many tokens per training example but without statistically significant improvements in performance. Consult Appendix C for details.

For a detailed account of our spectrogram construction and pre-processing, please consult Appendix A.

2.3 Tokenization

Recall that the Transformer is a sequence-to-sequence (Seq2Seq) model. Whether we represent EEG signals as multi-channel waveforms or spectrograms, we need to convert them into sequences that the Transformer can process. The process of turning pre-processed waveforms/spectrograms into Transformer-ready sequences is called tokenization. Whether it is for waveforms or spectrograms, the first step of tokenization is patching. This involves splitting the waveform/spectrogram into non-overlapping and equally-sized patches. Using non-overlapping patches instead of individual timesteps leads to a reduction in memory and runtime of attention map computation by a factor proportional to the stride squared (Nie et al., 2023). Each waveform is sliced into non-overlapping patches along the time axis. For spectrograms, we proceed with two options. The first option is to also take advantage of the time axis for each spectrogram. This involves slicing the spectrogram into groups of time bins. The second choice is to follow the approach of vision models like ViT, SimMIM, ViTMAE and ChannelViT. This involves slicing each spectrogram into square patches, thereby yielding separation in both the time and frequency axes. We illustrate square patching and time bin patching in Figure 2.1.

2.3.1 Channel-Mixing Projection

Whether it is waveforms or spectrograms, we then project each patch onto an embedding dimension of our choice. For ViT-Base (ViT-B), the embedding dimension is equal to 768. A key question is how to handle different channels during this projection. For RGB images, the typical choice is mixing channel information in each token (see section 1.5). With EEG signals, which can have upwards of 64 channels each representing a specific region of the brain, it is vital for the model to discern the role of each channel.

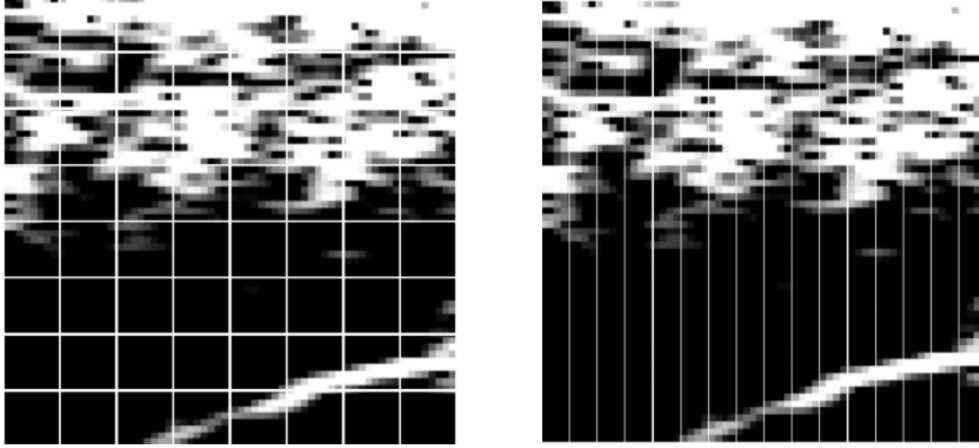


Figure 2.1: Spectrogram square patching (left) and time bin patching (right)

2.3.2 Channel-Independent Projection

We propose using a channel-independent projection such that 1 token only contains information from 1 channel. This projection was proposed by PatchTST for timeseries and by ChannelViT for images. While channel-independent projection has been studied in EEG waveforms, we are unaware of any existing method that has applied it to EEG spectrograms. We note that the primary limitation of channel-independent projection is that it yields more tokens per training example. Compared to channel-mixing, a channel-independent projection leads to an increase in the number of tokens, by a factor equal to the number of channels per training example. For EEG signals, where one can have anywhere between 19 and 64 channels, this becomes very significant, especially because the attention mechanism has quadratic time and space complexity in the number of input tokens.

In summary, whether it is waveforms or spectrograms, we first perform patching, then project each patch to a fixed embedding dimension in a channel-independent manner.

2.3.3 Positional and Channel Encodings

We add sinusoidal positional and channel encodings to our input tokens. The positional encodings uniquely identify each token's corresponding patch position in a given channel. The positional encodings are not learnable and they are shared across all channels. They are not learnable because the waveforms and spectrograms from different datasets have different position-specific quirks that we do not want to overfit to during pre-training. The channel encodings uniquely identify each token's corresponding channel. They are not learnable because we want to avoid learning channel-specific information during pre-training. We do this in order to promote generalizability downstream.

2.3.4 [CLS] Token

Typical models like ViTMAE and SimMiM, usually pre-pend a [CLS] token to each input sequence. This is a vector with the same dimensionality as each individual token. During pre-training and fine-tuning, the [CLS] token interacts with every other token via the attention mechanism and effectively contains a representation of all other tokens. In the downstream forward pass, a linear classification layer can be appended to the [CLS] token to facilitate the mapping from encoder representations to output classes. Without this [CLS] token one can use the mean of all contextual embeddings that are produced by the encoder. This has been shown to yield similar performance as using a [CLS] token (He et al., 2021). We design attention mechanisms that benefit from using the mean pooling strategy instead of a [CLS] token (see section 2.4).

2.4 Model Architecture

We use Transformer models (Vaswani, 2017) due to their demonstrated performance across a wide variety of EEG-specific tasks. Unless specified differently, we use a ViT-B model with 12 transformer blocks, 12 attention heads per block and a hidden dimension of 768, leading to a total of 85.5 million parameters.

2.4.1 Standard Self-Attention

Bahdanau (2014) introduced the original attention mechanism to solve the information bottleneck and sequence alignment issues in Seq2Seq models for neural machine translation. Its main strengths lie in enabling a model to attend to more relevant parts of the input and less on irrelevant parts depending on the learning task.

The equations below (equations 2.2 - 2.5) describe the multi-head self-attention mechanism as introduced by Vaswani (2017). Q , K and V (query, key and vector matrices) are obtained via a linear projection over the input X . Using these matrices, we can compute an attention score for each pair of tokens in the input, followed by a projection back to the original dimension via multiplication with the V matrix (see equation 2.3). Each head computes attention separately using a different set of projection parameters, leading to each head having the dimension d_e/h , where d_e is the embedding dimension and h is the number of attention heads per Transformer block. After concatenating the output from each head, the result is projected back to the embedding dimension d_e . We henceforth refer to this as *default* or *standard* self-attention.

$$Q, K, V = XW^Q, XW^K, XW^V \quad (2.2)$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2.3)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (2.4)$$

$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (2.5)$$

2.4.2 Split Attention

Recall that the standard self-attention mechanism is computed between every pair of tokens. This results in quadratic time and memory complexity in the number of input tokens. For EEG data, we hypothesize that this is unnecessary. For example, two tokens from channels that are far from

each other, representing time frames that are also very far from each other, do not necessarily need direct attention computation. Instead, we believe we can take advantage of the underlying structure of our tokens to promote indirect (and therefore more memory and time efficient) context communication.

To this end, we introduce *split attention*. Instead of pairwise attention computation, we aim to have intra-channel attention and inter-channel computation. Intra-channel computation should only occur between tokens within the same channel and should allow the model to learn local temporal dynamics. Inter-channel attention only occurs between tokens in the same time frame across different channels and should allow the model to learn inter-channel spatial correlations. We devised three candidates to achieve this in different ways: *Alternating* attention (algorithm 1), *Two Axis* attention (algorithm 2), and *Bottleneck* attention (algorithm 3).

Alternating Attention

Alternating attention (algorithm 1) involves performing one type of attention - either intra-channel or inter-channel attention - depending on the current index of the encoder block. We note that there is always an unused dimension. For example, intra-channel attention means there is an unused channel dimension. Inter-channel attention means there is an unused token position dimension. In both cases, the unused dimension is merged with the batch dimension. This allows us to leverage PyTorch parallelization instead of using explicit loops. Therefore, if an input tensor X has (B, Cp, D) where B is the batch size, C = number of channels, p is the number of patches per channel and D = hidden dimension, inter-channel attention over C will have shape (Bp, C, D) , and likewise, intra-channel attention over p will have dimensions (BC, p, D) . We use an even number of encoder blocks such that both intra-channel and inter-channel attention are equally represented.

Algorithm 1 ALTERNATING ATTENTION

Require: Input tensor X of shape $[B, Cp, d_e]$, where:

B : batch size

C : number of channels

p : number of patches per channel

d_e : embedding dimension

```

1: for each attention block index  $i$  do
2:   if  $i$  is odd then
3:     Reshape  $X$  to  $[Bp, C, d_e]$ 
4:     Compute QKV projection
5:     Compute Attention(Q, K, V)
6:   else
7:     Reshape  $X$  to  $[BC, p, d_e]$ 
8:     Compute QKV projection
9:     Compute Attention(Q, K, V)
10:  end if
11:  Reshape output back to  $[B, Cp, d_e]$ 
12: end for
```

Ensure: Output tensor of shape $[B, Cp, d_e]$

Two Axis Attention

In two axis attention (algorithm 2) we compute intra-channel and inter-channel attention in each encoder block, using two separate QKV projections. Unlike in alternating attention, each encoder layer contributes to the model learning the spatio-temporal dynamics of the input EEG signals. The primary limitation is that the additional QKV projection in each encoder layer adds more parameters and therefore complexity to the model. For example, with an embedding dimension of 768, 12 encoder layers and 12 attention heads in each encoder layer, two axis attention yields approximately 20M trainable parameters more than alternating attention.

Algorithm 2 TWO AXIS ATTENTION

Require: Input tensor X of shape $[B, Cp, d_e]$, where:

B : batch size

C : number of channels

p : number of patches per channel

d_e : embedding dimension

- 1: **for** each attention block **do**
- 2: Compute two QKV projections: Q_c, K_c, V_c and Q_p, K_p, V_p
- 3: Reshape X to $[Bp, C, d_e]$
- 4: Compute $A1 = \text{Attention}(Q_c, K_c, V_c)$
- 5: Reshape X to $[BC, p, d_e]$
- 6: Compute $A2 = \text{Attention}(Q_p, K_p, V_p)$
- 7: Reshape $A1$ and $A2$ to $[B, C, p, d_e]$
- 8: Compute $\text{Mean}(A1, A2)$
- 9: **end for**

Ensure: Output tensor of shape $[B, Cp, d_e]$

Bottleneck Attention

Bottleneck attention is our most experimental variation. It expands upon Two Axis, with the goal of having both intra-channel and inter-channel attention computation in each encoder block without resorting to 2 separate QKV projections. The idea is to first compute inter-channel attention and use the resulting attention map as a value vector in the intra-channel attention computation. Details are discussed in algorithm 3.

2.4.3 Handling the [CLS] Token

Notice that our implementations of split attention do not use any [CLS] tokens. During fine-tuning, we mean pool the encoder output tokens. From a theoretical perspective, it is also not clear how to incorporate the [CLS] token in separate intra-channel and inter-channel computations. We tried one implementation where the same [CLS] token was used for intra-channel and inter-channel attention computations, but found that it resulted in several additional tensor operations (e.g. permutations, expansions), which unnecessarily increase memory consumption and runtime.

Algorithm 3 BOTTLENECK SPLIT ATTENTION**Require:** Input tensor X of shape $[B, Cp, d_e]$, where: B : batch size C : number of channels p : number of patches per channel d_e : embedding dimension1: **for** each attention block **do**2: Compute QKV projection: Q, K, V of shape $[B, C, p, d_e]$ 3: $(QKV)_{avg(p)} = \text{MeanPool}(QKV)$ over p dimension, resulting in shape $[B, C, 1, d_e]$ 4: Compute $A1 = \text{Attention}(Q_{avg(p)}, K_{avg(p)}, V_{avg(p)})$ 5: $(QK)_{avg(C)} = \text{MeanPool}(QK)$ over C dimension, resulting in shape $[B, 1, p, d_e]$ 6: Broadcast($A1$) to shape $[B, C, p, d_e]$ 7: Compute $\text{Attention}(Q_{avg(C)}, K_{avg(C)}, A1)$ 8: **end for****Ensure:** Output tensor of shape $[B, Cp, d_e]$

2.4.4 Theoretical and Empirical Analysis of Split Attention

We posit that our three split attention candidates are more memory. We demonstrate this first by showcasing the theoretical memory complexity of each attention type in Table 2.1.

Attention Type	Memory Complexity	Time Complexity
Alternating (over p)	$\mathcal{O}(Cp^2)$	$\mathcal{O}(Cp^2d_e)$
Alternating (over C)	$\mathcal{O}(C^2p)$	$\mathcal{O}(C^2pd_e)$
Bottleneck	$\mathcal{O}(Cp(C + p))$	$\mathcal{O}(Cp(C + p)d_e)$
Two Axis	$\mathcal{O}(Cp(C + p))$	$\mathcal{O}(Cp(C + p)d_e)$
Standard	$\mathcal{O}((Cp)^2)$	$\mathcal{O}((Cp)^2d_e)$

Table 2.1: Theoretical memory and time complexities of each attention type.

We run experiments to measure the mean maximum GPU memory usage and mean runtime with increasing batch size. This mean is measured over 20 batched forward passes per model. We provide error bars to indicate the standard deviation in measurements. In each run, a ViT-B model is used. We measure results on 4 RTX 2090i GPUs. We conduct 2 sets of experiments. In figures 2.2 and 2.3 we show memory and runtime comparisons with 460 tokens per training example. This simulates tokenization of waveforms with $C = 23$ and $p = 20$. In this regime, we see that split attention (specifically alternating attention) yields up to $\sim 1.5X$ less GPU memory usage and comparable runtime to standard self-attention. In figures 2.4 and 2.5 we show memory and runtime comparisons with 1472 tokens per training example. This simulates tokenization of spectrograms with square patches, where $C = 23$ and $p = 64$. In this regime, we see that split attention (specifically alternating attention) yields up to $\sim 4X$ less GPU memory usage and $\sim 6X$ quicker runtime compared to standard self-attention. The empirical analysis suggests that split attention more efficient as the number of input tokens and batch size grow. This is corroborated by the theoretical analysis: we see in Table 2.1 that split attention is more efficient as the ratio $\frac{Cp}{(C+p)}$ grows. This is specifically relevant for EEG foundation models, where we can have > 64 channels (and therefore many tokens) and batch sizes on the order of 10^3 . We note that there

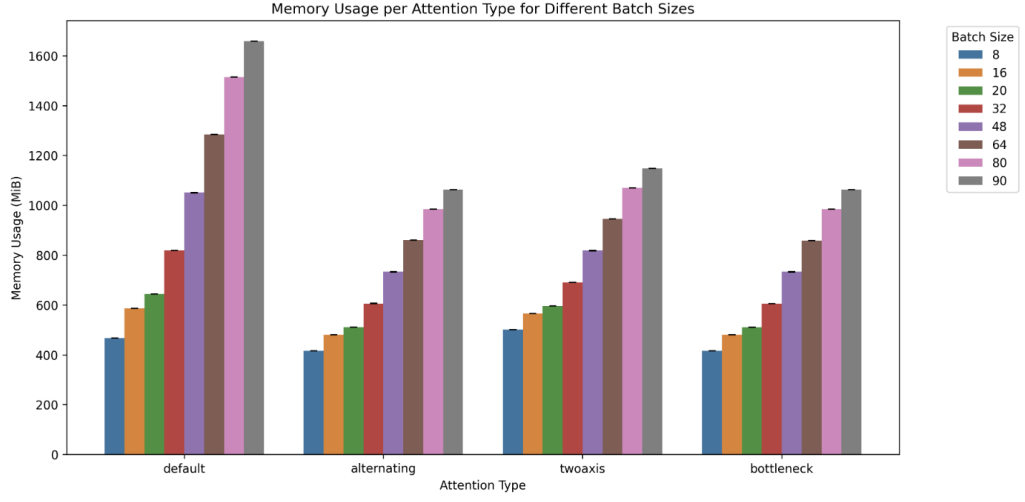


Figure 2.2: Mean model memory usage during inference for different attention mechanisms. In this regime, $C = 23$ and $p = 20$ which means we have 460 tokens per training example. This regime corresponds to waveform patching.

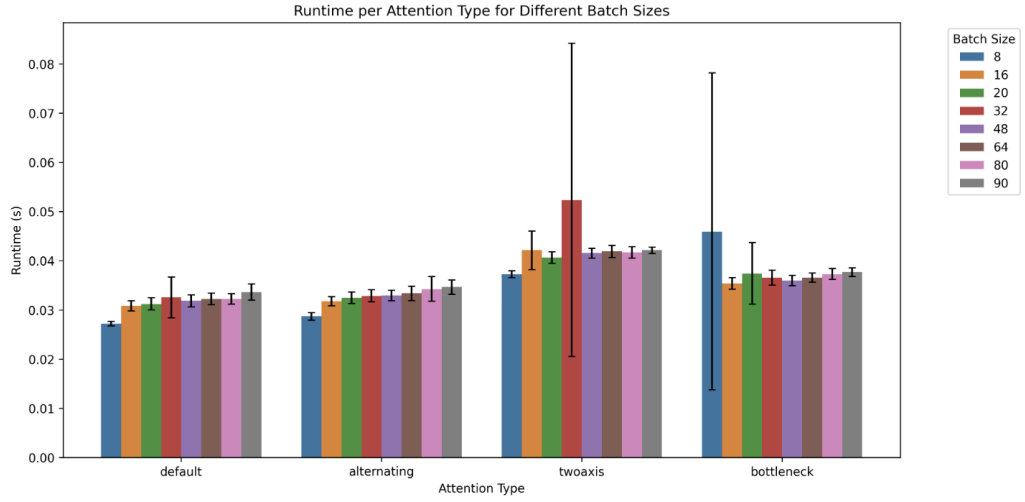


Figure 2.3: Mean model inference runtime for different attention mechanism. In this regime, $C = 23$ and $p = 20$ which means we have 460 tokens per training example. This regime corresponds to waveform patching.

exist efficient alternatives to self-attention, including but not limited to low-rank approximation methods (Chen et al., 2021), linear attention (Katharopoulos et al., 2020) and sparse attention (Lou et al., 2024). These methods typically provide memory and/or runtime improvements at the cost of performance compared to standard self-attention. This appears to be true for split attention applied to EEG spectrograms. However, we find that split attention on waveforms can actually provide performance improvements compared to standard self-attention (see section 4.3). Split attention can be combined with other self-attention alternatives (e.g. low rank approximations, linear attention etc) for potentially greater efficiency gains. Split attention can also, by design, be

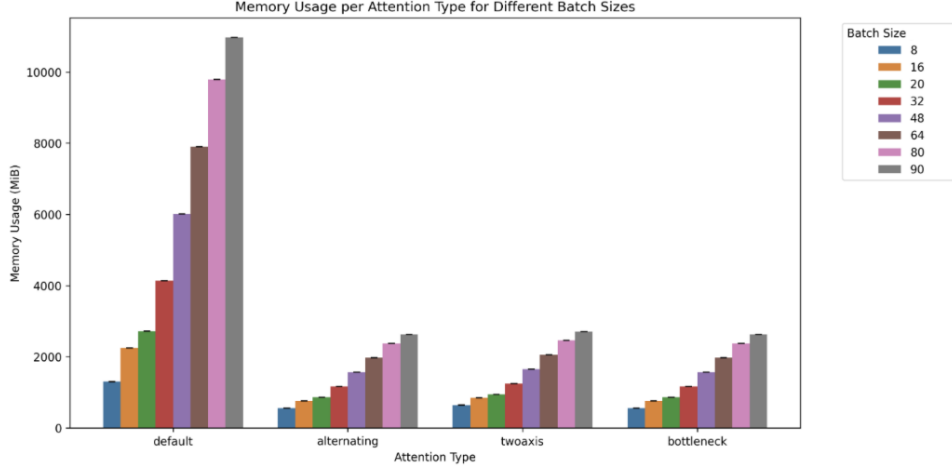


Figure 2.4: Mean model memory usage during inference for different attention mechanisms. In this regime, $C = 23$ and $p = 64$ which means we have 1472 tokens per training example. This regime corresponds to square patching of spectrograms.

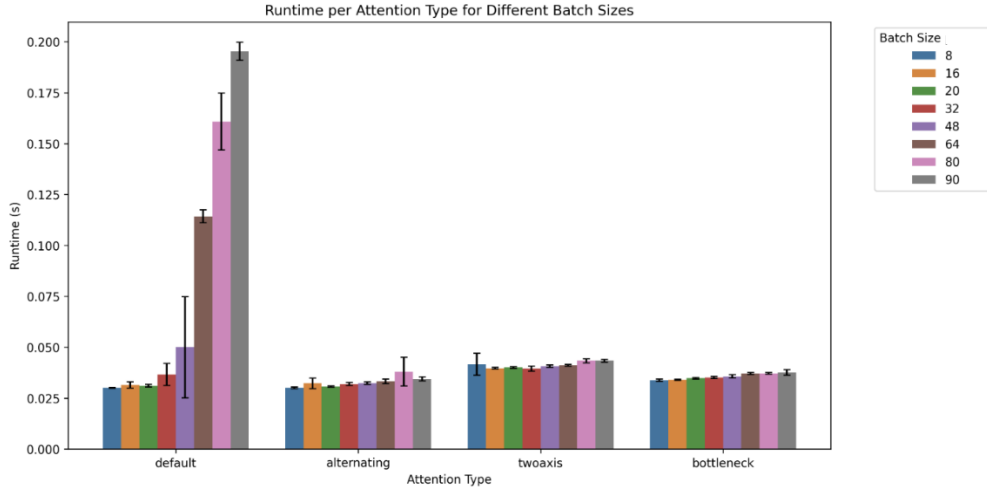


Figure 2.5: Mean model inference runtime for different attention mechanism. In this regime, $C = 23$ and $p = 64$ which means we have 1472 tokens per training example. This regime corresponds to square patching of spectrograms.

extended to general multi-channel timeseries and multi-channel images (although performance may vary by task and context). We leave these explorations for future research.

We illustrate our end-to-end pipeline for waveforms in Figure 2.6 and for spectrograms in Figure 2.7. Note that the same pipeline apply both in pre-training and fine-tuning. Due to the differences in output for pre-training and fine-tuning, the linear layer can have a variable number of trainable parameters.

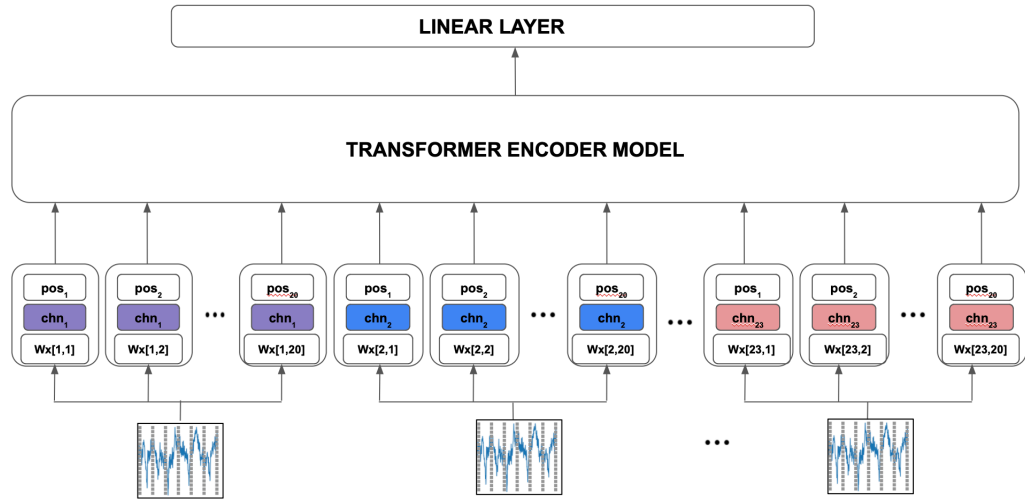


Figure 2.6: Our end-to-end pipeline for waveforms.

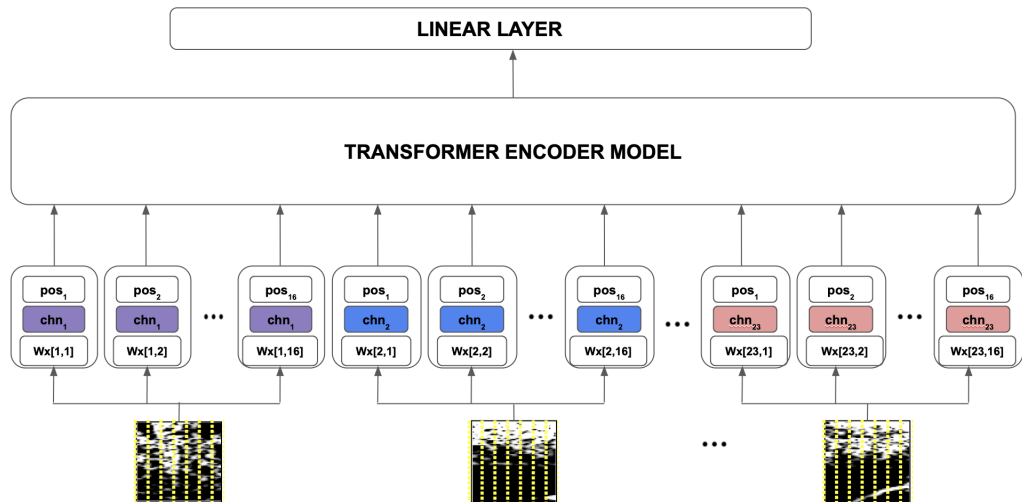


Figure 2.7: Our end-to-end pipeline for spectrograms.

2.5 Pre-training Strategy

2.5.1 Masking

We proceed with MAE as a pre-training task. This is done for several reasons. Firstly, most high performing vision, timeseries and EEG foundation models use the same task. The other popular alternative would be to use an invariance-based method like contrastive learning. In this regime, the objective is to train an encoder to generate similar embeddings for two or more views of the same training example (Weng et al., 2024). These different views are manually generated via augmentations. Such methods are popular in natural language processing and vision because of the nature of the data itself. For example, vision tasks benefit from a wide array of augmentations like rotation, zooming, blurring, and cropping among dozens of others (Mumuni et al., 2024). Text-related tasks benefit from augmentations like deletion, back-translation and random swaps (Feng et al., 2021). We posit that these augmentations are successful because they do not change the underlying semantic meaning of a given image or word. While there are augmentations available for EEG waveforms and spectrograms, such as SpecAugment (Park et al., 2019) and interpolation, it is not clear how much semantic meaning they preserve. This is partly because EEG spectrograms and waveforms are more difficult for humans to interpret compared to text or digital images. Inappropriate augmentations may generate synthetic artifacts inconsistent with authentic brain signals or even destroy crucial correlations (Ling et al., 2022), leading to a biased model.

Whether it is EEG waveforms or spectrograms, we implement a random masking strategy. This involves randomly masking a fixed portion of available tokens in each training example. When a token is masked, it is replaced by a dedicated [MASK] token. This is a learnable token that is shared across all masked positions. For spectrograms, we find that a masking ratio of $\sim 70\%$ is optimal. This is consistent with current literature. For waveforms, we find that a masking ratio of $\sim 50\%$ is optimal.

2.5.2 Pre-training Style

We experiment with two MAE styles: SimMiM and ViTMAE pre-training styles. Both are MAE styles that are well studied in literature. ViTMAE has even been applied to videos (Feichtenhofer et al. (2022)), single-channel audio spectrograms (Huang et al. (2022)) and cellular images (Kraus et al. (2023)). We are unaware of any method that has applied and compared these two methods in the context of EEG modelling, whether it be spectrograms or waveforms. We summarize the differences between these two styles in Table 2.2. For a detailed comparison, please consult subsection 1.5.2. In section 4.1, we observe very little differences between SimMiM and ViTMAE. However, we find that our split attention mechanism lends itself naturally to the SimMiM paradigm, where the encoder takes all tokens as input instead of just the visible ones. With SimMiM, the encoder can benefit from split attention both during pre-training and fine-tuning.

Pre-training Style	Encoder Input	Decoder Architecture	Decoder Input
SimMiM	Visible tokens & [MASK] tokens	Linear layer	Encoder output
ViTMAE	Visible tokens	Transformer decoder	Encoder output & [MASK] tokens

Table 2.2: Key differences between SimMiM and ViTMAE-style pre-training

2.5.3 Loss Function

Regardless of our representation and pre-training style, we train our model to reconstruct masked patches by minimizing the following loss function:

$$\mathcal{L}_{\text{overall}} = \mathcal{L}_{\text{masked_patches}} + \alpha \cdot \mathcal{L}_{\text{visible_patches}} \quad (2.6)$$

For waveforms, we find that the L_2 loss yields the best performance on TUAB. This is consistent with current literature (Nie et al., 2023; Zhang et al., 2023; Cui et al., 2024). For spectrograms, we find that a smooth L_1 loss yields the best downstream performance on TUAB. Future research could investigate other types of loss functions. MAE methods like ViTMAE, SimMiM and even BERT only compute the loss on masked patches (i.e. $\alpha = 0$). We observed that doing so in our model led to high quality reconstruction of the masked patches, but relatively poorer quality and generally unpredictable reconstructions of visible patches. We incorporated a loss term on visible patches and weighted it with $\alpha = 0.1$ to mitigate the unpredictable behavior of visible patch reconstruction all while preventing the model from learning the identify function during pre-training. A similar approach was used by Tan (2024) in the context of small scale RGB images like CIFAR10 ((Krizhevsky (2009)). We illustrate some pre-training reconstructions in Appendix E.

2.6 Fine-tuning Strategy

When we pre-train a model, we fix a number of epochs (see Appendix D). After each training epoch we run validation on a disjoint validation set. After the full training period, we select the checkpoint with the lowest pre-training validation loss. In practice there is no perfect linear correlation between validation loss and downstream performance, but it is the most principled and widely used method in the current literature. We then fine-tune our model on the downstream tasks presented in subsection 2.1.2. We proceed with full fine-tuning, where we train the pre-trained encoder and the classifier layer jointly, thereby using the pre-trained weights as initialization. Another alternative would be to use linear probing, where the pre-trained encoder is frozen and only the classifier layer are updated during downstream training. Given a carefully selected training recipe, full fine-tuning generally yields better results than linear probing whether it is in natural language processing, vision or timeseries (see methods discussed in section 1.5). We also add a training from scratch benchmark to illustrate the importance of pre-training compared to training downstream with random initialization. Our results are detailed in section 4.

The TimeFM Repository

3.1 What is TimeFM?

TimeFM is a Github repository that facilitates pre-training and fine-tuning of timeseries foundational models in a research context. It is designed to facilitate future research in the ETH Zurich Computer Vision Lab and Integrated Systems Lab.

3.1.1 PyTorch Lightning

TimeFM uses PyTorch Lightning (Falcon and The PyTorch Lightning Team, 2019) to drive all the components that govern model training. This is an open-source Python library that provides a high-level interface for PyTorch code. It is very popular among researchers since it allows them to focus purely on model mechanics while automating some engineering complexities. PyTorch Lightning is favored over PyTorch because it prevents the need for boiler-plate code. This leads to code that is generally more compact, understandable and reproducible. Finally, PyTorch Lightning preserves PyTorch's flexibility while allowing researchers to customize any part of the training process.

3.1.2 Hydra

In our case, training a single model involves a careful selection of dozens of hyperparameters and function arguments. This is typical especially in research that involves large neural networks. While PyTorch Lightning deals with the deep learning mechanics, we use Hydra (Yadan, 2019) to manage our experiment configurations. Hydra is an open-source library that allows us to clearly define experimental conditions in yaml files and therefore have control over every hyperparameter/possible arguments for every experiment. With Hydra, we can quickly iterate over many experiments.

3.2 Original State of TimeFM

We got access to TimeFM on March 25th 2024. At this point, TimeFM was a skeleton with enough code to show us what general functionality should look like. This skeleton is what we can be found in the TimeFM main branch <https://github.com/ofsoundof/TimeFM/tree/main>.

3.3 Current State of TimeFM

Since gaining access to the skeleton version of TimeFM, we have made very many modifications. We highlight them below. For an updated view of the TimeFM repository including all our code updates, please consult https://github.com/ofsoundof/TimeFM/tree/split_attention_foundation_model.

3.3.1 Criteria

One critical component of training a neural network is what we call the "criterion". This component defines exactly what is being optimized in a given model. For example, during our MAE pre-training, we have a reconstruction criterion in which we can compute an L1, L2 or smooth L1 loss between original and reconstructed spectrogram or waveform patches, as well as log the them to compare qualitatively. In this case, the corresponding Hydra configuration controls how to distribute the loss computed on visible and masked patches, whether we are using spectrograms or waveforms, whether to use square or time bin patches for spectrograms etc. For fine-tuning, we have a cross entropy criterion in which we can compare either binary or multi-class cross-entropy between true and predicted labels. In this case, the corresponding Hydra configuration controls aspects such as how much label smoothing to apply.

3.3.2 Schedulers

A learning rate scheduler is another important component for training state of the art neural networks. This functions controls the value of the learning over the whole training period. In this case, depending on which scheduler is used, the corresponding Hydra configuration controls the base learning rate and how long a warmup learning rate should be used (if it is used).

3.3.3 Datasets

We have specific Hydra configuration files that control the creation of our PyTorch dataset classes. In this case, the corresponding Hydra configuration controls the path to a given dataset, which subjects to consider for Leave-One-Subject-Out cross validation etc.

3.3.4 Data Modules

We define data modules to be modules that handle the creation and management of PyTorch data loaders, taken over from the aforementioned dataset classes. In this case, the corresponding Hydra configuration controls the path to a given dataset, what batch size to use and how many workers to use for each data loader for example.

3.3.5 Models

In our pipeline, we use the term "model" to specifically mean an "encoder". A model is perhaps the most important part of the training process. In this case, the corresponding Hydra configuration controls hyperparameters such as how large each patch should be, which attention mechanism to use and how deep the model should be among other hyperparameters.

3.3.6 Model Heads

In our pipeline, we use the term "model head" to specifically mean an "decoder" or prediction/reconstruction head that can be attached to an encoder. Since our project deals exclusively with foundation models, model heads are key. In this case, the corresponding Hydra configuration controls hyperparameters such as the input and output dimensionality of the model head and how deep the model head should be among other hyperparameters.

3.3.7 Pre-processors

Open-source EEG data typically comes with well accepted pre-processing steps (e.g. resampling, notch filtering and bandpass filtering). We perform this type of pre-processing only once when creating and saving each of our training examples. Although such steps can be performed on-the-fly (e.g. in a PyTorch data hook), we choose to perform these only once and save the result on disk in order to save time. Recall that our experiments are being performed on large models ($\sim 10^7$ training parameters) with large dataset ($\sim 10^6$ training examples). By default, our input data contains only pre-processed waveforms. We define additional "pre-processors" to be scripts that extract the time-frequency representation (e.g. computation of STFT). In this case, the corresponding Hydra configuration controls all STFT hyperparameters such as window length, hop length and whether to normalize each spectrogram.

3.3.8 Tasks

PyTorch Lightning allows users to explicitly control what happens in each training step, validation and test step. In our pipeline, we have separate "tasks" that control the training, validation and test steps, any data hooks (e.g. to compute STFT before a forward pass or perform any augmentations), optimizer and learning scheduler configuration, as well as the logging of any data and/or performance metrics. In this case, the corresponding Hydra configuration controls how aggressive any augmentations should be, whether to use any learning rate decay and whether to freeze the encoder model among other aspects.

Experiments

In this section, we conduct the main experiments of our report. These experiments provide us with empirical evidence to construct a "best" version of our model. In each experiment, we pre-train our model as discussed in section 2.1 on TUSZ, TUSL and TUEP. We then fine-tune our model on TUAB validation set. Note that the TUAB validation set is a random subset comprising 20% of the official TUAB training set. During model selection, we do not use the official TUAB test set. We follow the typical approach used by methods like ViTMAE and SimMiM, which carried out several model selection experiments on ImageNet-1K. Ideally, model selection should be performed by pre-training and fine-tuning the model on many different tasks and with a wide variety of model configurations (e.g. model sizes). We find our method to be a convenient proxy considering the relatively restricted scope of a Master project as well as the huge resources required to pre-train and fine-tune such models on mid-size to large datasets.

Unless specified otherwise, all experiments are conducted with a ViT-B encoder model. We perform pre-training and fine-tuning on a cluster of 8 RTX 4090 GPUs. For more details regarding all training hyperparameters and configurations, consult Appendix D.

4.1 Waveforms vs. Spectrograms

In this section, we compare pre-training and fine-tuning with waveforms and spectrograms. We compare both data representations with SimMiM and ViTMAE pre-training styles. Our experiment results are summarized in Table 4.1.

Representation	Pre-training Style	Balanced Accuracy (%)	AUPR	AUROC
Waveforms	None	76.39	0.8430	0.8426
	SimMiM	79.35	0.8755	0.8700
	ViTMAE	79.42	0.8741	0.8715
Spectrograms	None	76.49	0.8462	0.8483
	ViTMAE	77.71	0.8586	0.8559
	SimMiM	78.33	0.8659	0.8595

Table 4.1: The effect of EEG representation and pre-training on the TUAB downstream task.

4.2 Tokenization Strategy

In this section, we compare the use of square patches and time bin patches in spectrogram tokenization. Note that this also means we are comparing random unstructured masking with time bin masking. Results are outlined in Table 4.2.

Patching	Pre-training Style	Balanced Accuracy (%)	AUPR	AUROC
Square patches	None	76.49	0.8462	0.8483
Time bin patches	None	77.24	0.8489	0.8498
Square patches	SimMIM	78.33	0.8659	0.8595
Time bin patches	SimMIM	78.89	0.8719	0.8690

Table 4.2: The effect of patch shape and pre-training masking strategy on the TUAB downstream task.

4.3 Attention Mechanisms

In this section, we take the best waveform and spectrogram models from sections 4.1 and 4.2 and examine the what effect split attention mechanisms can have on the model’s downstream performance on TUAB. We consider only the SimMiM pre-training style. The ViTMAE pre-training style benefits less from split attention due to the fact that, as mentioned in sections 1.5 and 2.4, the input for the encoder consists only of visible tokens. Our results are highlighted in Table 4.1. We use each attention mechanism both in pre-training and fine-tuning.

Representation	Attention	Balanced Accuracy (%)	AUPR	AUROC
Spectrogram	Bottleneck	77.49	0.8548	0.8545
	Alternating	78.49	0.8684	0.8674
	Two Axis	78.83	0.8702	0.8671
	Standard	78.89	0.8719	0.8690
Waveform	Bottleneck	78.20	0.8640	0.8594
	Standard	79.35	0.8755	0.8700
	Two Axis	80.57	0.8885	0.8820
	Alternating	80.65	0.8901	0.8825

Table 4.3: The effect of split attention in pre-training and fine-tuning on the TUAB downstream task.

4.4 Comparisons to the State of the Art

Following our model selection experiments, we select our best performing model (using waveform representation and alternating attention) for comparison against the current state of the art

in 2 different downstream tasks. We pre-train and fine-tune 2 variants of our model, one with 85.5M parameters and another with 46.2M parameters. Both models are pre-trained on TUSZ, TUEP, TUEV, TUAR and TUSL (see Appendix B). The first downstream task is for our model to classify multi-channel EEG signals as either normal or abnormal. This is facilitated by the TUAB (TUH EEG Abnormal Corpus), in which there are official training and evaluation set. For this task, the two output classes are approximately balanced in both training and evaluation sets. Although TUAB is a subset of the TUH EEG Corpus, there is absolutely no overlap between the data or subjects in TUAB and the data or subjects seen in pre-training. The second downstream task is for our model to detect seizures from multi-channel EEG signals measured in newborn infants. This is facilitated by the Neonatal seizure dataset. Following state of the art practice, we measure performance using a 4-fold cross validation strategy. The Neonatal seizure dataset is heavily imbalanced, with only 13% of signals containing seizures. For each downstream task, we fine-tuned our model using 3 random seeds and report the mean and standard deviation in performance.

In each table, we directly report competing model sizes and performance as they appear in their original or competitor papers. Whenever values are missing in the table, it is because they are unavailable in any paper. We contacted all listed authors for information. Some authors (e.g. LaBraM authors) were responsive, but others were not.

Methods	Model Size	Balanced Accuracy (%)	AUPR	AUROC
Supervised Models				
SPaRCNet	0.79M	78.96 ± 0.18	0.8414 ± 0.0018	0.8676 ± 0.0012
ContraWR	1.6M	77.46 ± 0.41	0.8421 ± 0.0104	0.8456 ± 0.0074
CNN-Transformer	3.2M	77.77 ± 0.22	0.8433 ± 0.0039	0.8461 ± 0.0013
FFCL	2.4M	78.48 ± 0.38	0.8448 ± 0.0065	0.8569 ± 0.0051
ST-Transformer	3.5M	79.66 ± 0.23	0.8521 ± 0.0026	0.8707 ± 0.0019
Self-supervised Models				
BENDR	0.39M	76.96 ± 3.98	-	0.8397 ± 0.0344
BrainBERT	43.18M	-	0.8460 ± 0.0030	0.8530 ± 0.0020
EEGFormer-Small	-	-	0.8620 ± 0.0050	0.8620 ± 0.0070
EEGFormer-Base	-	-	0.8670 ± 0.0020	0.8650 ± 0.0010
EEGFormer-Large	-	-	0.8720 ± 0.0010	0.8760 ± 0.0030
BIOT	3.2M	79.59 ± 0.57	0.8792 ± 0.0023	0.8815 ± 0.0043
EEG2Rep	-	80.52 ± 2.22	-	0.8843 ± 0.0309
LaBraM-Base	5.8M	81.40 ± 0.19	0.8965 ± 0.0016	0.9022 ± 0.0009
LaBraM-Large	46M	82.26 ± 0.15	0.9130 ± 0.0005	0.9127 ± 0.0005
LaBraM-Huge	369M	82.58 ± 0.11	0.9204 ± 0.0011	0.9162 ± 0.0016
Ours	46.2M	80.41 ± 0.36	0.8912 ± 0.0007	0.8850 ± 0.0028
Ours	85.5M	80.40 ± 0.05	0.8884 ± 0.0022	0.8810 ± 0.0026

Table 4.4: Comparison of state of the art methods on TUAB.

Method	Model Size	AUPR	AUROC
Supervised Models			
EEGNet	-	0.499 ± 0.044	0.793 ± 0.019
TCN	-	0.398 ± 0.025	0.731 ± 0.020
EEG-GNN	0.107M	0.419 ± 0.021	0.760 ± 0.010
GraphS4mer	0.266M	0.374 ± 0.013	0.719 ± 0.007
STATENET	-	0.789	0.910
Self-supervised Models			
BrainBERT	43.18M	0.398 ± 0.027	0.734 ± 0.019
EEGFormer Small	-	0.578 ± 0.023	0.842 ± 0.008
EEGFormer Base	-	0.568 ± 0.036	0.842 ± 0.014
EEGFormer Large	-	0.544 ± 0.026	0.833 ± 0.017
Ours	46.2M	0.703 ± 0.098	0.883 ± 0.045
Ours	85.5M	0.716 ± 0.076	0.889 ± 0.034

Table 4.5: Comparison of state of the art methods on the Neonate seizure dataset.

Discussion

Our experiments reveal several interesting results. In Table 4.1, we observe that pre-training yields better performance than training from scratch. This can be seen across all 3 performance metrics for spectrograms and waveforms. One thing to note is that the difference is relatively small. Similar results are observed in other works (He et al., 2021; Xie et al., 2022). One could further improve the performance of the pre-training model by having a more meticulous hyperparameter search, a larger and more diverse pre-training dataset and carefully selected model size to avoid overfitting downstream. SimMiM and ViTMAE pre-training styles perform similarly in both regimes, with SimMiM yielding better performance in spectrograms while ViTMAE yields better performance in waveforms.

In Table 4.2, we observe that time bin patching yields better downstream performance on TUAB compared to the classical square patching used in standard vision tasks. We note that time bin patching is both superior in the training from scratch and pre-trained regimes. We posit that this is partly due to the sequential nature of tokens when time bin patching is used. This result is surprising considering that square patching (and therefore random unstructured masking) has shown superior performance in audio spectrogram tasks (Huang et al., 2022).

In Table 4.3, we observe distinct results in spectrograms compared to waveforms. For spectrograms, standard self-attention yields the most competitive results across all three metrics, closely followed by two axis and alternating attention. In contrast, two axis and alternating attention outperform standard self-attention for waveforms. In both regimes, bottleneck split attention is considerably worse than standard self-attention. We also observe that the best waveform representation yields significant improvements (notably +1.75% in the balanced accuracy, +0.0182 in the AUPR and +0.0135 in the AUROC) compared to the best spectrogram representation. Overall, a waveform representation seems to yield better performance across all metrics compared to a spectrogram representation. A thorough (but computationally intensive) hyperparameter search could mitigate this to some extent.

In Table 4.4, we see that our two models are quite competitive compared to the state of the art on the TUAB downstream dataset. In particular, our two models surpass all supervised models and self-supervised models in all performance metrics with the exception of EEG2Rep and LaBraM variations. We note that all LaBraM models are pre-trained on 1.7X more data than our model, whereas EEG2Rep is pre-trained on 16X more data than our model. In Table 4.5, we see our models are still superior to all BrainBERT and EEGFormer variations across all performance metrics. For approximately the same model size, our model yields a $\sim 70\%$ improvement in the AUPR compared to BrainBERT. Our worst performing model yields a $\sim 20\%$ improvement in the AUPR compared to the best performing EEGFormer model. This performance difference comes

despite EEGFormer being pre-trained on 16X more data than our model. In both TUAB and Neonate, we do however note that much smaller supervised models are still powerful. For example, STATENET surpasses all self-supervised models (including ours) across all metrics. We note that all self-supervised models are variants of the Transformer architecture, which is known to be relatively weak for time series (and more generally tabular) data ([Zeng et al., 2022](#); [Grinsztajn et al., 2022](#)).

Conclusion

We make several novel contributions in this Master project. As far as we know, we propose the first comparison of spectrogram and waveform representations in a single EEG foundation model. We are also the first foundation model to compare the ViTMAE and SimMiM MAE pre-training styles. Furthermore, we propose a novel tokenization scheme for multi-channel EEGs and demonstrate.

To mitigate the computational overhead introduced by our tokenization scheme, we propose 3 novel self-attention variants, which we coin split attention. We provide theoretical and empirical memory and runtime analyses to illustrate the merits of our split attention mechanisms compared to standard self-attention. We demonstrate that our split attention mechanisms are particularly useful in cases where we have many tokens ($\sim 10^3$) per training example and large batch sizes. In such cases we show that split attention can reduce memory usage by a factor of ~ 4 and runtime by a factor of ~ 6 . Our split attention mechanism not only significantly reduces memory usage and computational time, but it can also improve downstream performance when compared to standard self-attention.

The combination of these components leads to the proposal of an entirely novel EEG foundation model. We pre-train two variants of our model on a heterogeneous pre-training dataset, totalling over 1400 hours of EEG recordings. We measure our model's performance on 2 downstream tasks: anomaly classification and neonatal seizure detection. The datasets for these two tasks contain patient profiles unseen during pre-training. Despite having a relatively small pre-training dataset, our models are superior than most state of the art methods in direct comparisons, only slightly surpassed by methods with significantly larger pre-training corpora.

Finally, we develop TimeFM, a Github repository to facilitate training of timeseries foundation models.

In summary, our theoretical and experimental contributions open up several research avenues - some of which we intend to pursue (see section 7) - and our implementation of TimeFM opens the door for other students and researchers to iterate upon our work and extend further into other use cases.

Limitations and Future Work

As with all scientific works, our work has limitations. We distinguish between the limitations of our algorithmic contributions and the limitations of our experiments.

On the algorithmic side, we note that our tokenization is limited to a fixed number of pre-determined channels. In practice, different EEG recordings can have a variable number of channels. We will dedicate our future work to accounting for this. In particular, we are considering three alternatives. Inspired by NLP, we can use channel padding. This involves padding the input token sequence with zero-valued channels (or a special [PAD] token) up to a fixed maximum number of channels. This would allow our model to process a pre-determined range of channel numbers. One potential limitation is that this method may bias the model to ignore relatively important channels. Inspired by Neuro-GPT, we can also use nearest neighbor interpolation to infer any missing channels up to a pre-defined maximum. The limitation is that we may lose fine-grained information in individual channels. Finally, we also propose a novel electrode-aware pooling strategy. In the International 10-20 system, EEG channel nomenclature is specific. Channel names typically consist of letters and numbers that indicate the electrode's location on the scalp. Consider the channel name "FP2". The first letters ("FP" in this case) denote the general brain area (frontal pole in this case). The number ("2" in this case) indicates the hemisphere and relative position (right hemisphere in this case). We can pool electrodes both by general brain area and by hemisphere according to the channel nomenclature. This pooling is anatomically consistent and can, in principle, accommodate any number of channels. However, it likely that we lose fine-grained detail (but perhaps less than we would if we used interpolation methods). We can further promote channel variations by implementing pre-training augmentations. One example is random channel dropping.

When medical experts diagnose brain conditions, EEG signals alone are insufficient. Proper diagnosis often requires more patient data including but not limited to family history, patient clinical history, imaging studies (MRI, PET and/or CT scans) among others. This multi-modal approach should also be reflected in any algorithms that are designed to fully automate clinical diagnosis. In practice, training such a multi-modal model would be challenging not only from a tokenization perspective, but also from a data access perspective.

Our novel split attention mechanism also shows promise, both in terms of its efficiency and performance compared to standard self-attention. However, our model may struggle to capture long-range or lagged inter-channel correlations. One direction for future research could be to form a more hierarchical split attention structure, such that earlier encoder layers focus on intra-channel and short-term inter-channel relationships and later layers incorporate more long-term inter-channel correlations.

On the experimental side, we are limited by the amount of data used both for pre-training and

evaluation. In fact, this is a weakness of in most of the state of the art methods discussed in 1.5. In pre-training, while we do use a subset of the TUH EEG corpus, it still likely does not capture the majority of inter-subject variability one could have in a single country, let alone the whole world. We evaluate our model on 2 downstream tasks, but this still does not cover the wide range of clinically relevant downstream tasks (e.g. sleep staging, emotion recognition). This is perhaps appropriate given that that this report is done in the context of a Master project with 2 students. However, the sheer promise of our results calls for future work. We are aiming to write a scientific paper based on the research done in our Master project. To meet the higher scientific standard, we aim to use a much larger pre-training corpus (e.g. the full TUH EEG corpus with 27,000+ hours of EEG recordings) and more variation in channel configurations. To demonstrate the full capability of our model, we aim to examine its performance in many more downstream tasks such as sleep stage classification, emotion recognition and complex motor imagery among others. Furthermore, state of the art comparisons in EEG foundation models are very difficult because competing methods use different sizes, downstream tasks and pre-training corpora. Without the resources and/or implementations to run all competing methods, we were limited in our choice of downstream tasks. However, new large scale and well-funded research is emerging. BrainWave (Yuan et al., 2024a) is also a new foundation model for general brain signals, developed by 8 researchers from 6 different laboratories worldwide. The authors establish rigorous benchmarks in which several established methods are compared in 15 downstream tasks. We aim to leverage this work to scrutinize our own model. We do not include BrainWave in our related works nor our state of the art comparisons, because it was released 1 week before our project submission and it lies outside of the scope of study since it is pre-trained on both intracranial and scalp EEG signals.

Pre-processing Details

For all datasets in the TUH EEG corpus, we resample signals to a sampling rate of 256 Hz, apply a bandpass filter with a low frequency of 0.1 Hz and a high frequency of 75 Hz. We use a notch filter at 60Hz to remove power line interference, and then slice the signals into non-overlapping 5 second samples. For TUAB, we preserve the original sampling rate of 250 Hz. This practice follows what is typically done in EEG literature.

For the Neonate dataset, they are already pre-processed (bandpass and notch filters) by the authors. For Neonate, we consider the majority vote from the 3 independent medical experts that annotate the data. We then slice the data into non-overlapping 5 second segments, labelled with 1 if there is any seizure within the window and 0 otherwise.

Given an input EEG signal that has been pre-processed sliced into non-overlapping equally-sized patches, we use an STFT transform with a Hann window of length 256, 256 time points for each FFT computation and a hop length of 20. The window length and number of time points for FFT correspond to the sampling frequency we use in pre-training. This allows us to have spectrograms with a 1 Hz frequency resolution and a 78 ms time resolution (Jain et al. (2015)). The 1 Hz frequency resolution allows clear separation of key EEG frequency bands like delta (0.5-4 Hz), theta (4-8 Hz), alpha (8-13 Hz), and beta (13-30 Hz). Due to the time-frequency resolution trade-off (see equation 2.1), we cannot precisely select our desired time resolution. The 78 ms time resolution has 2 benefits. Firstly, it falls within the average time taken for visual and audio stimuli to yield neural responses. This is important for our downstream tasks. Secondly, it balances the time-frequency resolution trade-off for our needs. Decreasing this value would compromise the frequency resolution. Increasing the value would mean significantly reducing the number of time bins available for processing. We log-transform and then z-score normalize the magnitude tensor in each training example.

In the waveform representation regime, we normalize and scale the signal in each patch to the range [-1, 1].

Dataset Statistics

We provide detailed dataset statistics for all the data used in this project.

Dataset	Number of Samples	Number of Channels	Sample Duration (s)	Sampling Frequency (Hz)
TUAR	66666	23	5	256
TUEV	72212	23	5	256
TUEP	303081	23	5	256
TUSL	16503	23	5	256
TUSZ	598944	23	5	256
TUAB	819561	23	5	256
Neonate	80536	19	5	256

Table B.1: Summary of EEG datasets used in our Master project

Ablation Studies

C.1 Incorporating Magnitude Information Into Spectrogram Tokenization

In Table C.1, we study the impact of combining magnitude and phase information when spectrograms are used as input to our model. Specifically, we evaluate the performance on the TUAB dataset. We introduce phase as additional "channels" in our spectrogram tensors.

In the interleaved approach, the "channel" dimension follows the structure:

$$[\text{magnitude}_1, \text{phase}_1, \text{magnitude}_2, \text{phase}_2, \dots, \text{magnitude}_{23}, \text{phase}_{23}]$$

where magnitude_i and $\text{phase}_i, i \in [1, 23]$ represent the pre-processed magnitude and phase tensors of channel i . In the separated approach, the "channel" dimension follows the structure:

$$[\text{magnitude}_1, \text{magnitude}_2, \dots, \text{magnitude}_{23}, \text{phase}_1, \text{phase}_2, \dots, \text{phase}_{23}]$$

We observe that including phase information doubles the number of tokens in each training example. This results in a $\sim 4X$ increase in runtime and memory for self-attention computation. In practice, including phase information does not lead to statistically significant improvements in performance on TUAB. Note that Table C.1 illustrates results when training a ViT-B model from scratch on TUAB.

Spectrogram Information	Balanced Accuracy (%)
Magnitude only	77.21
Magnitude + interleaved phase	76.01
Magnitude + separated phase	76.89

Table C.1: Comparison of spectrogram magnitude vs. magnitude and phase information on TUAB.

C.2 Hyperparameter Search

Training a model as high in variance as a transformer is bound to lead to overfitting unless well regularized. This appendix describes the hyper-parameter settings attempted and justifies the

choices made throughout the process of optimizing our fine-tuning recipe for the task of anomaly detection on the TUAB dataset. Using the SimMiM fine-tuning recipe as our baseline and starting point, we focused primarily on reducing overfitting in our models. Thus, we conducted experiments using the following hyperparameters: batch size, learning rate, drop path and augmentation noise added. All other hyperparameters remained unchanged from the SimMiM setting (Xie et al., 2022). Since we aim to reduce overfitting in our model, emphasis is placed in particular on reducing the difference between the training metrics and the validation metrics.

Model #	Batch Size	LR	DropPath	Noise Amplitude	Training B. Acc. (%)	Val. B. Acc. (%)	Training-Val difference
1	4096	5e-4	0.0	0.2	95.97	78.52	17.45
2	4096	5e-4	0.1	0.2	90.42	79.12	11.29
3	4096	5e-4	0.2	0.2	88.85	79.66	9.18
4	4096	5e-4	0.0	0.5	94.99	77.65	17.34
5	4096	5e-4	0.2	0.5	87.81	78.69	9.11
6	4096	5e-4	0.1	0.5	89.07	78.61	10.46
7	2048	5e-4	0.2	0.2	92.75	79.32	13.42
8	2048	5e-4	0.25	0.1	91.76	79.51	12.24
9	2048	5e-5	0.2	0.2	81.09	79.35	1.73

Table C.2: Hyperparameter optimization: Fine-tuning SimMiM with patch size 64, masking 50% of tokens at epoch 101 using various hyperparameters. Note that all metrics follow the same trends displayed by the Balanced Accuracy metrics.

We put into comparison the following experiments:

- 1: baseline using simMiM hyperparameters, with the addition of white noise (0.2)
- 2 and 3: testing the effect of adding drop path as a form of regularization; we concluded that drop path has a strong regularization effect and we will henceforth use it
- 4, 5 and 6: testing the effect of adding white noise with larger or smaller amplitudes as a form of regularization, sometimes in addition with drop-path; we concluded that white noise helps in small amounts (0.2). Large amplitudes hinder both validation and training performance.
- 7, 8, 9: Primarily aimed at testing the effects of smaller batch size and smaller learning rate. 7 compares directly to the baseline 1, showing significant improvements across the board. 8 compares to 7 while varying the drop path and noise amplitude parameters, resulting in mild improvement. Finally 9 compares with 7 by reducing the learning rate. This final experiment is the only one to have competitive validation performance while also showing a similar training performance.

Following the analysis above, we concluded that the highest impact in our improvements was due to the addition of drop path, and, although reducing both batch size and learning rate showed the largest reduction in overfitting, we decided to use the hyperparameters which gave us the largest performance (parameters set 3).

Training Recipes

	Small Model (46.2M params)	Large Model (85.5M params)
Batch Size		4096
Scheduler		Cosine + Linear warmup
Optimizer		AdamW
Betas		[0.9, 0.98]
Peak learning rate		1.25e-3
Minimum learning rate		2.5e-7
Total epochs		100
Warmup epochs		10
Masking ratio		0.5
Encoder layers		12
Number of attention heads per layer		12
Embedding dimension	564	768
Weight decay		0.5
Patch size		64

Table D.1: Pre-training hyperparameters

	Small Model (46.2M params)	Large Model (85.5M params)
Batch Size		2048
Scheduler		Cosine + Linear warmup
Optimizer		AdamW
Betas		[0.9, 0.999]
Peak learning rate		5e-4
Minimum learning rate		2.5e-7
Total epochs		50
Warmup epochs		5
Layer-wise learning rate decay factor		0.8
Weight decay		0.5
Gaussian noise amplitude ratio		0.2
Drop path		0.1
Label smoothing		0.1

Table D.2: Fine-tuning hyperparameters

Pre-training Reconstructions

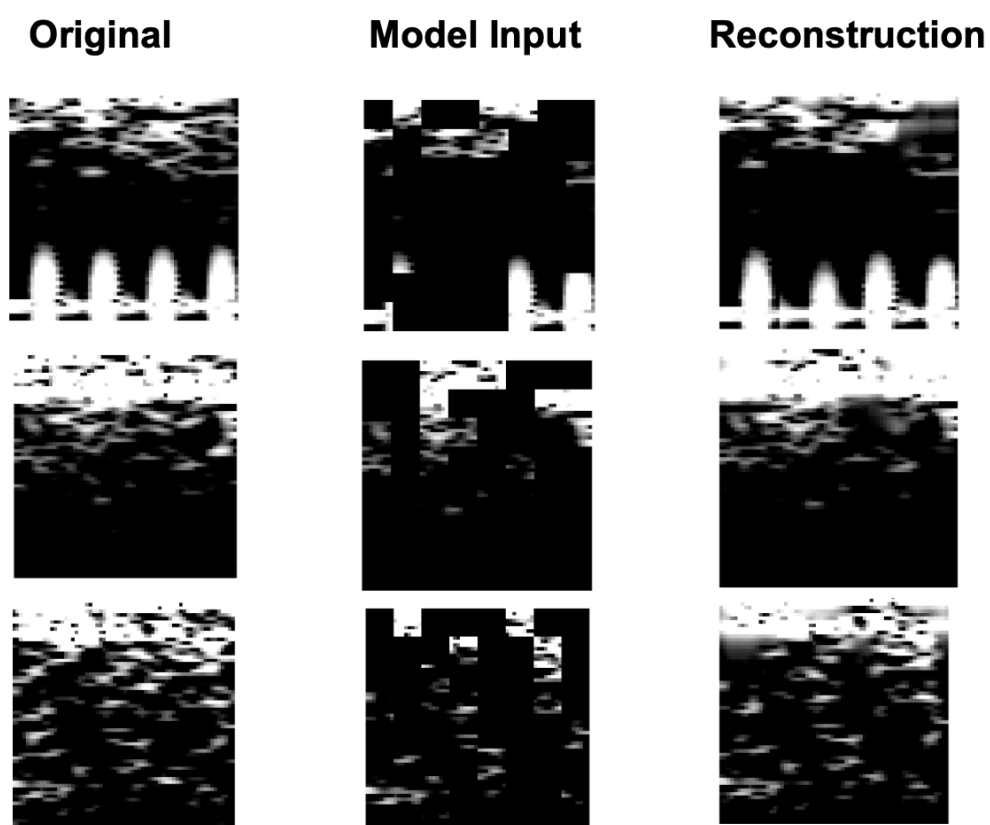


Figure E.1: Reconstructions of spectrograms with square patching.

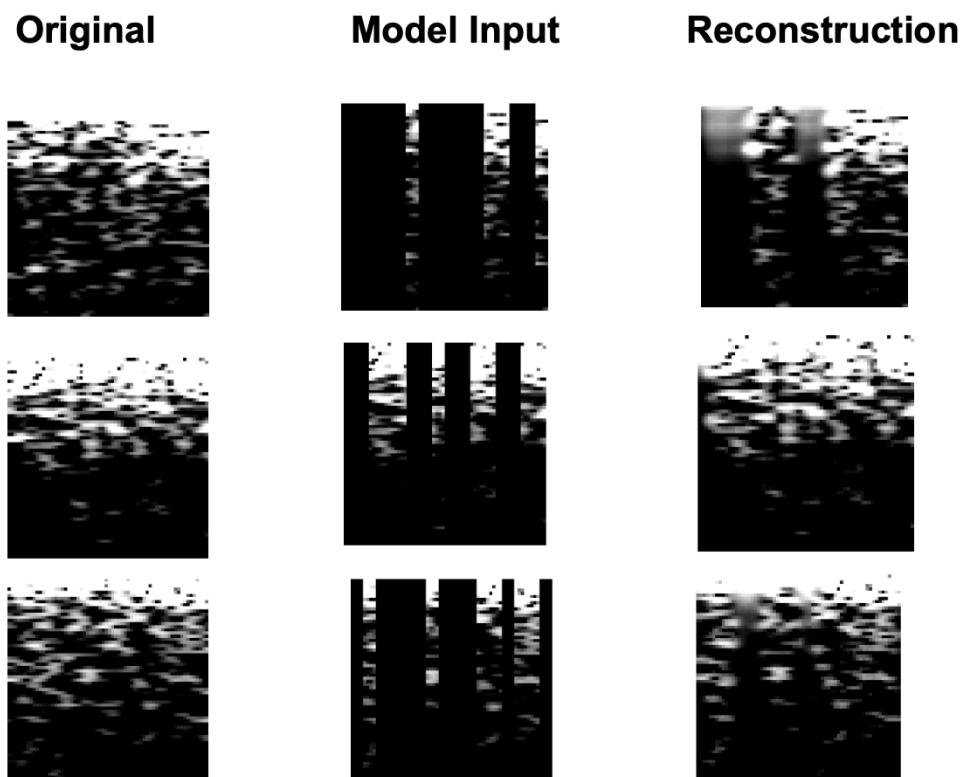


Figure E.2: Reconstructions of spectrograms with time bin patching.

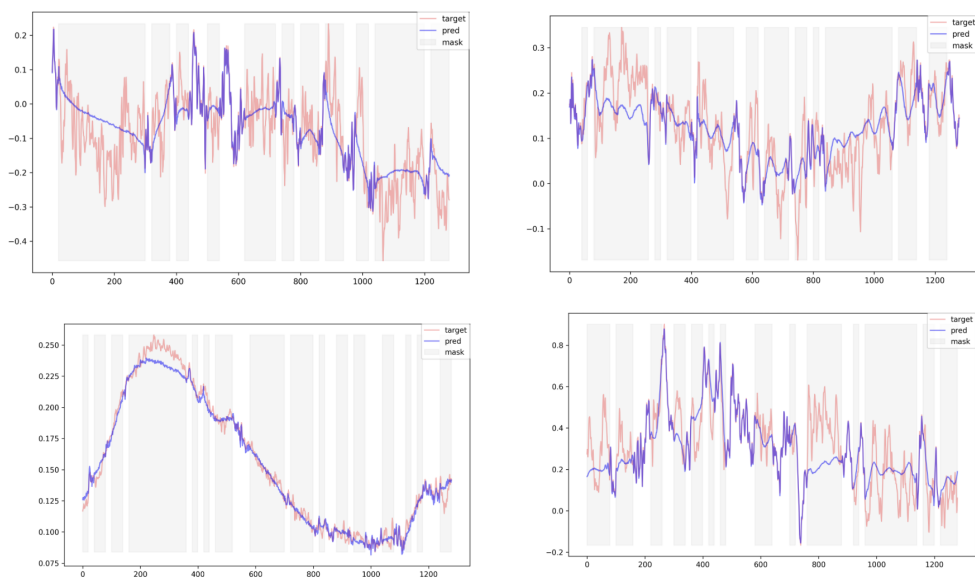


Figure E.3: Reconstructions of waveforms.

What Did Not Work

Throughout the project, we encountered several things that resulted in sub-optimal performance:

- We initially proceeded with a BrainBERT-like denoising auto-encoding pre-training task. We randomly selected time bins and frequency bins to be replaced by pre-determined out of distribution values. We note that the masking ratios were also quite low (less than 50% on average). We observed that pre-training and fine-tuned performance was not statistically different compared to training from scratch on TUAB.
- For the spectrogram patching module, we initially used a Conv3D layer similarly to the ChannelViT implementation. However, we observed that using a tensor reshape followed by a linear layer achieved the same result but 5X faster on average.
- Storage of such a large dataset was a challenge, particularly on machines where storage is shared among students at research staff at ETH Zurich. We initially stored all EEG recordings as individual .edf files, which would be parsed and read into memory. This was very slow, leading to approximately 10 minutes for data loading. We then stored each individual training example in .h5 format, resulting in data loading on the order of seconds. However, we quickly exhausted our quota on the number of files stored. Our final solution was to "concatenate" each the training example in .h5 format. This way, data loading is still on the order of seconds, but each dataset effectively occupies a single in the system.
- We experimented with various MLP and convolutional prediction heads for our model. These prediction heads brought not statistically significant performance improvement, but lead to an increase in the number of trainable parameters (and therefore an increase in the number of floating point operations (FLOPs)). Similarly to SimMiM, we found that a single linear layer is sufficient both for pre-training and fine-tuning.
- Spectrogram representations were quite tricky to pre-process. We initially used BrainBERT's frequency-wise normalization, but observed that it resulted in poor fine-tuning performance (our model was effectively no better than random chance). However, we noticed that BrainBERT is tailored specifically to iEEG signals where higher frequency information is very important. In contrast, we work with scalp EEGs where it is the low frequency signals that are important. We found that standard per-channel normalization led to a significant boost in our model's performance.
- We initially computed our pre-training loss exclusively on masked patches. In the ViTMAE regime, we observed that the behavior of the reconstructed visible patches was highly inconsistent. Adding the loss component on visible patches led to an overall decrease of the entire validation loss function, but also better downstream performance.

- We fine-tuned our model on the BCI IV 2a dataset. This is a small motor imagery dataset in which a model is supposed to classify between 4 motor imagery tasks (left hand movement, right hand movement, dual foot movement and tongue movement). The dataset is very small (< 5000 training examples). We observed that our model was highly prone to overfitting. We posit that a more appropriate training recipe (notably more aggressive regularization) would mitigate this, but we did not have the computational resources to conduct a thorough hyperparameter search for such a large model.

List of Figures

2.1	Spectrogram square patching (left) and time bin patching (right)	16
2.2	Mean model memory usage during inference for different attention mechanisms. In this regime, $C = 23$ and $p = 20$ which means we have 460 tokens per training example. This regime corresponds to waveform patching.	21
2.3	Mean model inference runtime for different attention mechanism. In this regime, $C = 23$ and $p = 20$ which means we have 460 tokens per training example. This regime corresponds to waveform patching.	21
2.4	Mean model memory usage during inference for different attention mechanisms. In this regime, $C = 23$ and $p = 64$ which means we have 1472 tokens per training example. This regime corresponds to square patching of spectrograms.	22
2.5	Mean model inference runtime for different attention mechanism. In this regime, $C = 23$ and $p = 64$ which means we have 1472 tokens per training example. This regime corresponds to square patching of spectrograms.	22
2.6	Our end-to-end pipeline for waveforms.	23
2.7	Our end-to-end pipeline for spectrograms.	23
E.1	Reconstructions of spectrograms with square patching.	49
E.2	Reconstructions of spectrograms with time bin patching.	50
E.3	Reconstructions of waveforms.	50

List of Tables

2.1	Theoretical memory and time complexities of each attention type.	20
2.2	Key differences between SimMiM and ViTMAE-style pre-training	24
4.1	The effect of EEG representation and pre-training on the TUAB downstream task.	31
4.2	The effect of patch shape and pre-training masking strategy on the TUAB downstream task.	32
4.3	The effect of split attention in pre-training and fine-tuning on the TUAB downstream task.	32
4.4	Comparison of state of the art methods on TUAB.	33
4.5	Comparison of state of the art methods on the Neonate seizure dataset.	34
B.1	Summary of EEG datasets used in our Master project	43
C.1	Comparison of spectrogram magnitude vs. magnitude and phase information on TUAB.	45
C.2	Hyperparameter optimization: Fine-tuning SimMiM with patch size 64, masking 50% of tokens at epoch 101 using various hyperparameters. Note that all metrics follow the same trends displayed by the Balanced Accuracy metrics.	46
D.1	Pre-training hyperparameters	47
D.2	Fine-tuning hyperparameters	48

List of Listings

Bibliography

- Baevski, A., Zhou, H., Mohamed, A., and Auli, M. (2020). wav2vec 2.0: A framework for self-supervised learning of speech representations. In *Thirty-fourth Conference on Neural Information Processing Systems*.
- Bahdanau, D. (2014). Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*.
- Bao, H., Dong, L., Piao, S., and Wei, F. (2022). BEit: BERT pre-training of image transformers. In *International Conference on Learning Representations*.
- Bao, Y., Sivanandan, S., and Karaletsos, T. (2024). Channel vision transformers: An image is worth $1 \times 16 \times 16$ words. In *The Twelfth International Conference on Learning Representations*.
- Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., Bernstein, M. S., Bohg, J., Bosselut, A., Brunskill, E., Brynjolfsson, E., Buch, S., Card, D., Castellon, R., Chatterji, N., Chen, A., Creel, K., Davis, J. Q., Demszky, D., Donahue, C., Doumbouya, M., Durmus, E., Ermon, S., Etchemendy, J., Ethayarajh, K., Fei-Fei, L., Finn, C., Gale, T., Gillespie, L., Goel, K., Goodman, N., Grossman, S., Guha, N., Hashimoto, T., Henderson, P., Hewitt, J., Ho, D. E., Hong, J., Hsu, K., Huang, J., Icard, T., Jain, S., Jurafsky, D., Kalluri, P., Karamcheti, S., Keeling, G., Khani, F., Khattab, O., Koh, P. W., Krass, M., Krishna, R., Kuditipudi, R., Kumar, A., Ladhak, F., Lee, M., Lee, T., Leskovec, J., Levent, I., Li, X. L., Li, X., Ma, T., Malik, A., Manning, C. D., Mirchandani, S., Mitchell, E., Munyikwa, Z., Nair, S., Narayan, A., Narayanan, D., Newman, B., Nie, A., Niebles, J. C., Nilforoshan, H., Nyarko, J., Ogut, G., Orr, L., Papadimitriou, I., Park, J. S., Piech, C., Portelance, E., Potts, C., Raghunathan, A., Reich, R., Ren, H., Rong, F., Roohani, Y., Ruiz, C., Ryan, J., Ré, C., Sadigh, D., Sagawa, S., Santhanam, K., Shih, A., Srinivasan, K., Tamkin, A., Taori, R., Thomas, A. W., Tramèr, F., Wang, R. E., Wang, W., Wu, B., Wu, J., Wu, Y., Xie, S. M., Yasunaga, M., You, J., Zaharia, M., Zhang, M., Zhang, T., Zhang, X., Zhang, Y., Zheng, L., Zhou, K., and Liang, P. (2022). On the opportunities and risks of foundation models.
- Boudewyn, M. A., Erickson, M. A., Winsler, K., Ragland, J. D., Yonelinas, A., Frank, M., Silverstein, S. M., Gold, J., MacDonald, A. W. r., Carter, C. S., Barch, D. M., and Luck, S. J. (2023). Managing EEG studies: How to prepare and what to do once data collection has begun. *Psychophysiology*, 60(11):e14365.
- Brockwell, P. J. and Davis, R. A. (2016). *Introduction to Time Series and Forecasting*. Springer Texts in Statistics. Springer, Cham, 3 edition.
- Burle, B., Spieser, L., Roger, C., Casini, L., Hasbroucq, T., and Vidal, F. (2015). Spatial and temporal resolutions of eeg: Is it really black and white? a scalp current density view. *International Journal of Psychophysiology*, 97(3):210–220. Epub 2015 May 12.

- Caron, M., Touvron, H., Misra, I., Jégou, H., Mairal, J., Bojanowski, P., and Joulin, A. (2021). Emerging properties in self-supervised vision transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9650–9660.
- Chang, C., Wang, W.-Y., Peng, W.-C., and Chen, T.-F. (2024). Llm4ts: Aligning pre-trained llms as data-efficient time-series forecasters.
- Chen, B., Dao, T., Winsor, E., Song, Z., Rudra, A., and Ré, C. (2021). Scatterbrain: Unifying sparse and low-rank attention approximation.
- Chen, Y., Ren, K., Song, K., Wang, Y., Wang, Y., Li, D., and Qiu, L. (2024). Eegformer: Towards transferable and interpretable large-scale eeg foundation model.
- Cui, W., Jeong, W., Thölke, P., Medani, T., Jerbi, K., Joshi, A. A., and Leahy, R. M. (2024). Neurogpt: Towards a foundation model for eeg.
- Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. (2009). Imagenet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 248–255.
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). Bert: Pre-training of deep bidirectional transformers for language understanding.
- Diachenko, M., Houtman, S. J., Juarez-Martinez, E. L., Ramautar, J. R., Weiler, R., Mansvelder, H. D., Bruining, H., Bloem, P., and Linkenkaer-Hansen, K. (2022). Improved manual annotation of EEG signals through convolutional neural network guidance. *eNeuro*, 9(5):ENEURO.0160–22.2022.
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., and Houlsby, N. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*.
- El-Nouby, A., Izacard, G., Touvron, H., Laptev, I., Jegou, H., and Grave, E. (2021). Are large-scale datasets necessary for self-supervised pre-training?
- Falcon, W. A. and The PyTorch Lightning Team (2019). Pytorch lightning. <https://github.com/Lightning-AI/lightning>.
- Feichtenhofer, C., Fan, H., Li, Y., and He, K. (2022). Masked autoencoders as spatiotemporal learners. In Oh, A. H., Agarwal, A., Belgrave, D., and Cho, K., editors, *Advances in Neural Information Processing Systems*.
- Feichtinger, H. G. and Strohmer, T., editors (1998). *Gabor Analysis and Algorithms*. Applied and Numerical Harmonic Analysis. Birkhäuser, Boston, MA.
- Feng, S. Y., Gangal, V., Wei, J., Chandar, S., Vosoughi, S., Mitamura, T., and Hovy, E. (2021). A survey of data augmentation approaches for nlp.
- Foumani, N. M., Mackellar, G., Ghane, S., Irtza, S., Nguyen, N., and Salehi, M. (2024). Eeg2rep: Enhancing self-supervised eeg representation through informative masked inputs.
- Gherman, S., Markowitz, N., Tostaeva, G., et al. (2024). Intracranial electroencephalography reveals effector-independent evidence accumulation dynamics in multiple human brain regions. *Nature Human Behaviour*, 8:758–770.

- Grinsztajn, L., Oyallon, E., and Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on tabular data?
- Gu, A. and Dao, T. (2024). Mamba: Linear-time sequence modeling with selective state spaces.
- Gu, A., Goel, K., and Re, C. (2022). Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*.
- He, K., Chen, X., Xie, S., Li, Y., Dollár, P., and Girshick, R. (2021). Masked autoencoders are scalable vision learners.
- Huang, P.-Y., Xu, H., Li, J. B., Baevski, A., Auli, M., Galuba, W., Metze, F., and Feichtenhofer, C. (2022). Masked autoencoders that listen. In Oh, A. H., Agarwal, A., Belgrave, D., and Cho, K., editors, *Advances in Neural Information Processing Systems*.
- Hussein, R., Lee, S., and Ward, R. (2022). Multi-channel vision transformer for epileptic seizure prediction. *Biomedicines*, 10(7):1551.
- Jain, A., Bansal, R., Kumar, A., and Singh, K. (2015). A comparative study of visual and auditory reaction times on the basis of gender and physical activity levels of medical first year students. *International Journal of Applied and Basic Medical Research*, 5(2):124–127.
- Janiukstyte, V., Owen, T. W., Chaudhary, U. J., Hamandi, K., and Lemieux, L. (2023). Normative brain mapping using scalp EEG and potential clinical application. *Scientific Reports*, 13:13442.
- Jiang, W., Zhao, L., and liang Lu, B. (2024). Large brain model for learning generic representations with tremendous EEG data in BCI. In *The Twelfth International Conference on Learning Representations*.
- Jin, M., Wang, S., Ma, L., Chu, Z., Zhang, J. Y., Shi, X., Chen, P.-Y., Liang, Y., Li, Y.-F., Pan, S., and Wen, Q. (2024). Time-LLM: Time series forecasting by reprogramming large language models. In *The Twelfth International Conference on Learning Representations*.
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. (2020). Scaling laws for neural language models.
- Katharopoulos, A., Vyas, A., Pappas, N., and Fleuret, F. (2020). Transformers are rnns: Fast autoregressive transformers with linear attention.
- Klonowski, W. (2009). Everything you wanted to ask about eeg but were afraid to get the right answer. *Nonlinear Biomedical Physics*, 3(1):2.
- Kostas, D., Aroca-Ouellette, S., and Rudzicz, F. (2021). Bendr: using transformers and a contrastive self-supervised learning task to learn from massive amounts of eeg data.
- Kraus, O., Kenyon-Dean, K., Saberian, S., Fallah, M., McLean, P., Leung, J., Sharma, V., Khan, A., Balakrishnan, J., Celik, S., Sypetkowski, M., Cheng, C., Morse, K., Makes, M., Mabey, B., and Earnshaw, B. (2023). Masked autoencoders are scalable learners of cellular morphology. In *NeurIPS 2023 Generative AI and Biology (GenBio) Workshop*.
- Krizhevsky, A. (2009). Learning multiple layers of features from tiny images. Technical report, University of Toronto, Toronto, Canada.
- Lachaux, J.-P., Axmacher, N., Mormann, F., Halgren, E., and Crone, N. E. (2012). High-frequency neural activity and human cognition: past, present and possible future of intracranial eeg research. *Progress in Neurobiology*, 98(3):279–301. Epub 2012 Jun 26.

- Lee, H. H., Gu, Y., Zhao, T., Xu, Y., Yang, J., Usuyama, N., Wong, C., Wei, M., Landman, B. A., Huo, Y., Santamaria-Pang, A., and Poon, H. (2024). Foundation models for biomedical image segmentation: A survey.
- Lee, T. S. (1996). Image representation using 2d gabor wavelets. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 18(10):959–971.
- Ling, H., Luyuan, Y., Xinxin, L., and Bingliang, D. (2022). Staging study of single-channel sleep eeg signals based on data augmentation. *Frontiers in Public Health*, 10:1038742.
- Lou, C., Jia, Z., Zheng, Z., and Tu, K. (2024). Sparser is faster and less is more: Efficient sparse attention for long-range transformers.
- Moca, V., Bârzan, H., Nagy-Dăbâcan, A., and Mureşan, R. (2021). Time-frequency super-resolution with superlets. *Nature Communications*, 12.
- Mumuni, A., Mumuni, F., and Gerrar, N. K. (2024). A survey of synthetic data augmentation methods in machine vision. *Machine Intelligence Research*, 21(5):831–869.
- Nie, Y., Nguyen, N. H., Sinthong, P., and Kalagnanam, J. (2023). A time series is worth 64 words: Long-term forecasting with transformers. In *The Eleventh International Conference on Learning Representations*.
- Obeid, I. and Picone, J. (2016). The temple university hospital eeg data corpus. *Frontiers in Neuroscience*, 10:196.
- Panchavati, S. and Speier, W. (2024). Mentality: A mamba-based approach towards foundation models for eeg. In *ICLR 2024 Workshop on Learning from Time Series For Health*.
- Park, D. S., Chan, W., Zhang, Y., Chiu, C.-C., Zoph, B., Cubuk, E. D., and Le, Q. V. (2019). SpecAugment: A simple data augmentation method for automatic speech recognition. In *Interspeech 2019, interspeech2019.ISCA*.
- Parvizi, J. and Kastner, S. (2018). Promises and limitations of human intracranial electroencephalography. *Nature Neuroscience*, 21(4):474–483. Epub 2018 Mar 5.
- Ramon, C., Freeman, W. J., Holmes, M., Ishimaru, A., Haueisen, J., Schimpf, P. H., and Rezvanian, E. (2009). Similarities between simulated spatial spectra of scalp eeg, meg and structural mri. *Brain Topography*, 22(3):191–196. Epub 2009 Jun 26.
- Rashid, M., Bari, B. S., and Sadeque, M. G. (2016). Mathematical principals and modeling of EEG signal exploration. *IOSR Journal of VLSI and Signal Processing*, 6(6):05–09.
- Ronneberger, O., Fischer, P., and Brox, T. (2015). U-net: Convolutional networks for biomedical image segmentation.
- Roy, Y., Banville, H., Albuquerque, I., Gramfort, A., Falk, T. H., and Faubert, J. (2019). Deep learning-based electroencephalography analysis: a systematic review. *Journal of Neural Engineering*, 16(5):051001.
- Schirrmeister, R. T., Springenberg, J. T., Fiederer, L. D. J., Glasstetter, M., Eggensperger, K., Tangermann, M., Hutter, F., Burgard, W., and Ball, T. (2017). Deep learning with convolutional neural networks for EEG decoding and visualization. *Human Brain Mapping*, 38(11):5391–5420.
- Sejdić, E., Djurović, I., and Jiang, J. (2009). Time-frequency feature representation using energy concentration: An overview of recent advances. *Digital Signal Processing*, 19(1):153–183.

- Shoeb, A. (2010). CHB-MIT Scalp EEG Database. PhysioNet. Available at <https://physionet.org/content/chbmit/1.0.0/>.
- Stevenson, N. J., Tapani, K., Lauronen, L., and Vanhatalo, S. (2019). A dataset of neonatal eeg recordings with seizure annotations. *Scientific Data*, 6:190039.
- Tan, J. H. (2024). Pre-training of lightweight vision transformers on small datasets with minimally scaled images.
- Teplan, M. (2002). Fundamental of eeg measurement. *MEASUREMENT SCIENCE REVIEW*, 2.
- van den Oord, A., Vinyals, O., and Kavukcuoglu, K. (2017). Neural discrete representation learning. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*.
- Vaswani, A. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*.
- Wang, C., Subramaniam, V., Yaari, A. U., Kreiman, G., Katz, B., Cases, I., and Barbu, A. (2023). BrainBERT: Self-supervised representation learning for intracranial recordings. In *The Eleventh International Conference on Learning Representations*.
- Weng, W., Gu, Y., Guo, S., Ma, Y., Yang, Z., Liu, Y., and Chen, Y. (2024). Self-supervised learning for electroencephalogram: A systematic survey.
- Xie, Z., Zhang, Z., Cao, Y., Lin, Y., Bao, J., Yao, Z., Dai, Q., and Hu, H. (2022). Simmim: A simple framework for masked image modeling.
- Yadan, O. (2019). Hydra - a framework for elegantly configuring complex applications. Github.
- Yang, C., Westover, M. B., and Sun, J. (2023). Biot: Cross-data biosignal learning in the wild.
- Yuan, Z., Shen, F., Li, M., Yu, Y., Tan, C., and Yang, Y. (2024a). Brainwave: A brain signal foundation model for clinical applications.
- Yuan, Z., Zhang, D., Chen, J., Gu, G., and Yang, Y. (2024b). Brant-2: Foundation model for brain signals.
- Zeng, A., Chen, M., Zhang, L., and Xu, Q. (2022). Are transformers effective for time series forecasting?
- Zhai, X., Puigcerver, J., Kolesnikov, A., Riquelme, C., Bayer, M., Gelly, S., Oliver, A., Lucic, M., and Houlsby, N. (2019). A large-scale study of representation learning with the visual task adaptation benchmark. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3502–3511.
- Zhang, D., Yuan, Z., Yang, Y., Chen, J., Wang, J., and Li, Y. (2023). Brant: Foundation model for intracranial neural signal. In *Thirty-seventh Conference on Neural Information Processing Systems*.